DevOps, Cloud & Platform Engineering

Complete Notes: from zero to job-ready

Networking · Linux · Git · Bash · Python · AWS · Docker
Terraform · Ansible · CI/CD · Kubernetes · Helm · EKS
Monitoring · SRE · DevSecOps · GitOps · Platform Engineering · Interview Q&A

Thirupathi Keloth

100-Day Learning Program · 2026

Contents

1. Computer & IT Basics

2. Networking

3. Linux

4. Git & GitHub

5. Bash Scripting

6. Python for Automation

7. AWS Cloud

8. Docker

9. Terraform (Infrastructure as Code)

10. Ansible (Configuration Management)

11. CI/CD

12. Kubernetes

13. Monitoring & Observability

14. DevSecOps (Security Throughout)

15. GitOps & Argo CD

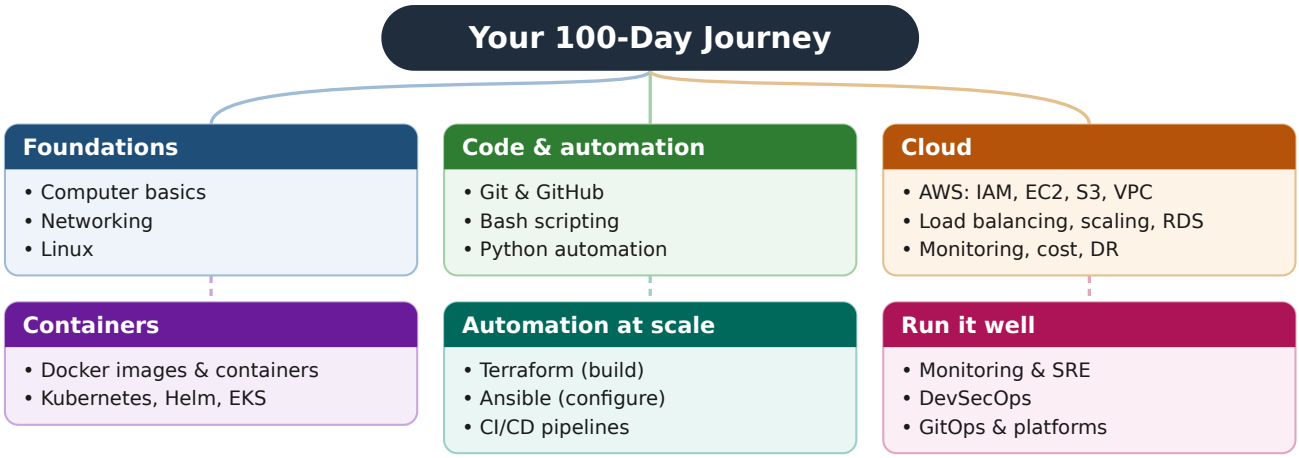
16. Platform Engineering

17. Troubleshooting Playbooks

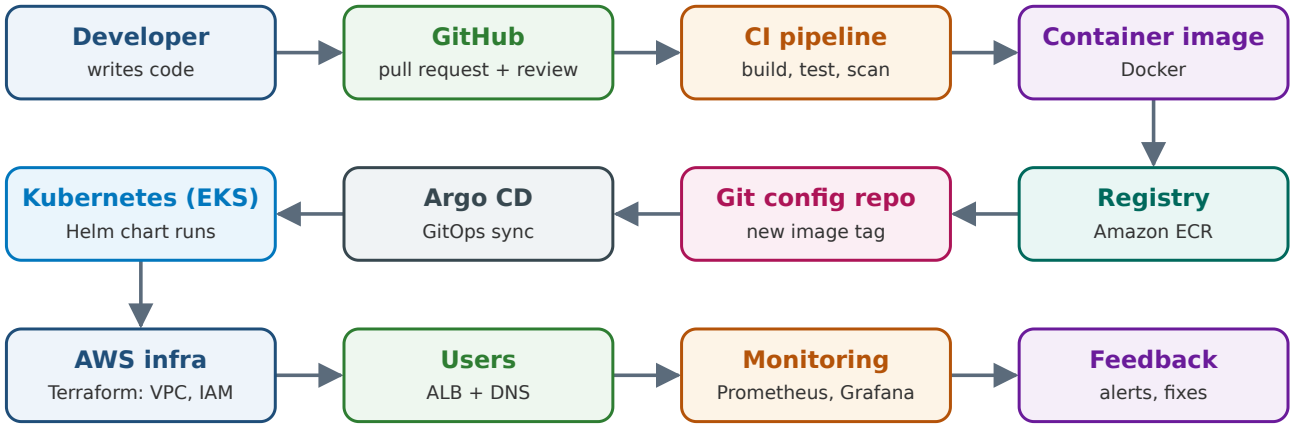
18. Interview Questions (Quick Answers)

Revision History

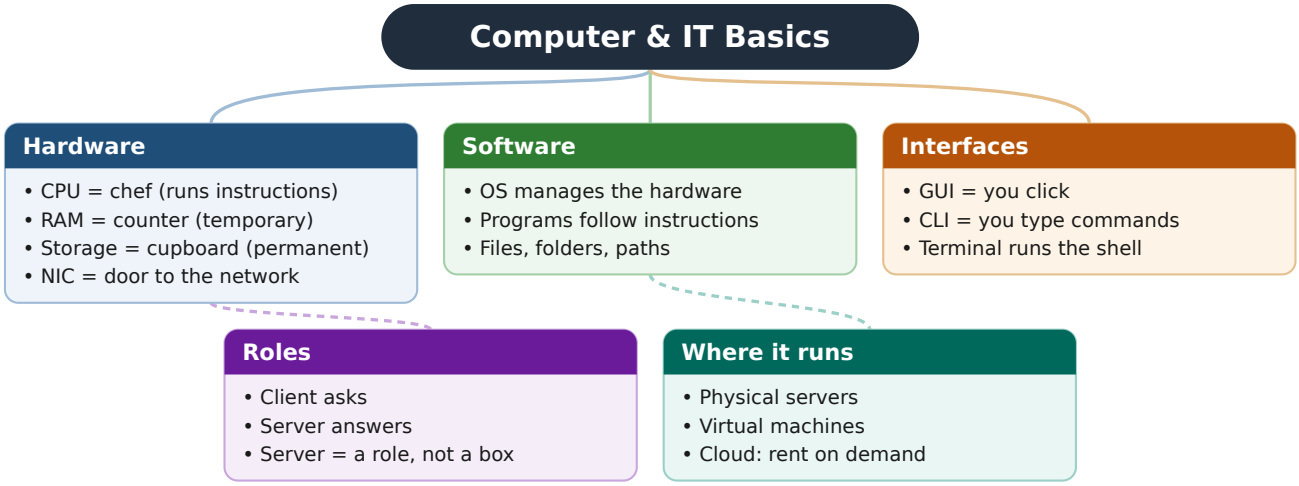
The Big Picture



How it all connects (your final project)



1. Computer & IT Basics



Core parts of a computer

Part	What it does	Remember it as
CPU	Runs instructions and calculations	The chef
RAM	Fast, short-term working memory. Cleared at power off	The kitchen counter
Storage (SSD/HDD)	Keeps files permanently	The cupboard
Network card (NIC)	Connects the computer to a network	The front door

Key terms

Term	Meaning
Operating system (OS)	The main program that manages hardware and runs apps (Windows, macOS, Linux)
Program / application	A set of instructions the computer follows
File / folder (directory)	Saved data / a container for files
Path	The address of a file. Windows: `C:\Users\me\file.txt`. Linux: `/home/me/file.txt`
GUI	Graphical interface: you click
CLI / terminal / shell	Text interface: you type commands
Client	The computer or program that asks for something
Server	The computer or program that answers. "Server" is a role, not a special machine
Virtual machine (VM)	A full computer simulated in software, running on a real one
Cloud	Renting computers, storage and services over the internet (AWS, Azure, GCP)

Why DevOps uses the terminal: servers have no screen, commands can be automated and repeated, and they are exact and fast.

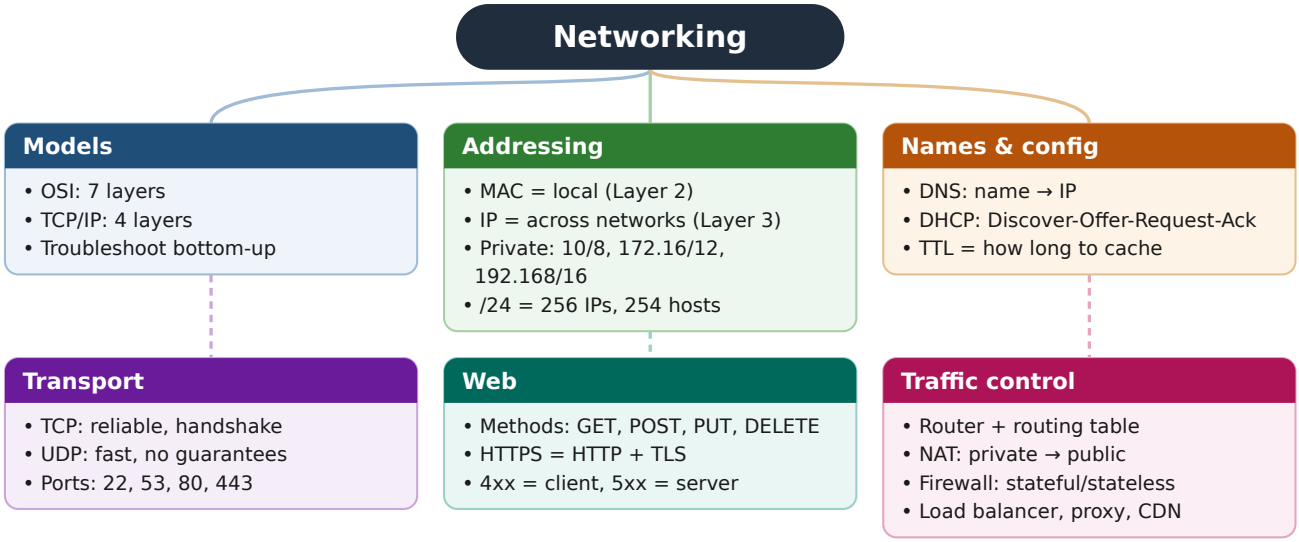
First Windows PowerShell commands

Command	What it does
`hostname`	Shows the computer's name
`whoami`	Shows the logged-in user
`ipconfig`	Shows your IP address and gateway
`ipconfig /all`	Full network details, including the MAC address and DNS servers
`ping google.com`	Tests whether another computer can be reached
`cls`	Clears the screen

REMEMBER

- Every computer: **Input → Process → Store → Output**.
- RAM is fast but forgets at power-off; storage keeps data.
- Servers usually run **Linux**, have no screen, and are managed through a **terminal** over a network.

2. Networking



2.1 How data travels

```
Client  ──request──►  Router  ──►  ISP  ──►  Internet  ──►  Server
        ◄──response──
```

- Data is split into small pieces called **packets**. Each packet carries a source and destination IP address.
- **Protocol** = an agreed set of rules for communicating (HTTP, TCP, DNS...).

2.2 OSI and TCP/IP models

OSI layer	TCP/IP layer	Job	Examples
7 Application	Application	What the user or app wants	HTTP, HTTPS, DNS, SSH, FTP
6 Presentation	Application	Format and encryption	TLS, JSON
5 Session	Application	Keeping conversations open	Sessions
4 Transport	Transport	Reliable or fast delivery, ports	TCP, UDP
3 Network	Internet	Addressing and routing	IP, ICMP, routers
2 Data link	Network access	Delivery inside the local network	MAC, Ethernet, switches, ARP
1 Physical	Network access	Bits on a cable or radio	Cables, Wi-Fi

Troubleshoot bottom-up: cable/Wi-Fi → IP address → gateway → DNS → port → application.

2.3 Devices and addresses

Term	Meaning
MAC address	A physical address burned into the network card, e.g. `00:1A:2B:3C:4D:5E`. Used inside the local network
IP address	A logical address used to reach a device across networks
Switch	Connects devices inside one network using MAC addresses
Router	Connects different networks and forwards packets using IP addresses
Default gateway	The router your device sends traffic to when the destination is outside your network
ARP	Finds the MAC address that belongs to an IP address on the local network

2.4 IPv4 addressing

- 32 bits, written as 4 numbers (0–255): `192.168.1.10`
- **Private ranges** (not reachable from the internet): `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`
- **Public IP:** unique on the internet, assigned by your ISP or the cloud provider
- **Special:** `127.0.0.1` = loopback (this machine, "localhost"); `169.254.x.x` = APIPA (DHCP failed); `0.0.0.0` = all addresses
- **IPv6:** 128 bits, e.g. `2001:db8::1`. Created because IPv4 ran out of addresses

Binary refresher (one octet):

Bit value	128	64	32	16	8	4	2	1
192 =	1	1	0	0	0	0	0	0

2.5 Subnets and CIDR

- A **subnet** is a smaller network carved out of a bigger one.
- **CIDR** `/n` = how many bits belong to the network part. The rest is for hosts.
- **Usable hosts** = $2^{(32 - n)} - 2$ (network address and broadcast address are reserved). AWS reserves 5 per subnet.

CIDR	Subnet mask	Total IPs	Usable hosts
/8	255.0.0.0	16,777,216	16,777,214
/16	255.255.0.0	65,536	65,534
/20	255.255.240.0	4,096	4,094
/24	255.255.255.0	256	254
/26	255.255.255.192	64	62
/28	255.255.255.240	16	14
/32	255.255.255.255	1	1 (a single host)

Example: `10.0.1.0/24` → network `10.0.1.0`, hosts `10.0.1.1–10.0.1.254`, broadcast `10.0.1.255`.

Typical cloud plan: VPC `10.0.0.0/16` → public subnets `10.0.1.0/24`, `10.0.2.0/24` → private subnets `10.0.11.0/24`, `10.0.12.0/24`.

How subnetting works (binary method)

Place values of one octet (memorize): 128 · 64 · 32 · 16 · 8 · 4 · 2 · 1. All eight bits on = 255.

Convert	Method	Example
Binary → decimal	Add the place values where the bit is 1	`10101100` = 128+32+8+4 = **172**
Decimal → binary	Left to right: if the number ≥ place value, write 1 and subtract; else 0	200 → 128 ✓ (72), 64 ✓ (8), 32 ✗, 16 ✗, 8 ✓ (0), 4 ✗, 2 ✗, 1 ✗ = `11001000`

The mask in binary: 1 = network bit, 0 = host bit. `/27` = `11111111.11111111.11111111.11100000` = 255.255.255.224 → 5 host bits → 2^5 = 32 addresses, 30 usable. **Network address** = IP AND mask (host bits set to 0). **Broadcast** = all host bits set to 1.

Fast block-size method:

1. Find the octet where the mask is not 255 or 0.
2. Block size = 256 − mask value in that octet.
3. Subnets start at multiples of the block size. The IP belongs to the largest multiple ≤ its value.
4. Broadcast = next subnet − 1.

Example `192.168.10.77/27`: mask .224 → block 32 → subnets .0, .32, **.64**, .96 → network `192.168.10.64`, hosts `.65`–`.94`, broadcast `.95`. **Hosts → prefix:** ≤14 → /28 · ≤30 → /27 · ≤62 → /26 · ≤126 → /25 · ≤254 → /24 · ≤510 → /23.

(Full step-by-step binary and subnetting practice: see the *Computer Networking* notes, section 6.)

2.6 DNS (Domain Name System)

Turns names into IPs: `google.com → 142.250.x.x`.

```
Browser → OS cache → Resolver (ISP / 8.8.8.8)
        → Root (.) → TLD (.com) → Authoritative server → IP returned
```

Record	Purpose
A	Name → IPv4 address
AAAA	Name → IPv6 address
CNAME	Name → another name (alias)
MX	Mail server for the domain
TXT	Text, e.g. domain verification, SPF
NS	Which name servers are authoritative for the domain
TTL	How long (in seconds) a record may be cached

`/etc/hosts` (Linux) or `C:\Windows\System32\drivers\etc\hosts` (Windows) overrides DNS locally.

2.7 DHCP

Gives devices an IP address, subnet mask, gateway and DNS server automatically. Process **DORA**: Discover → Offer → Request → Acknowledge.

2.8 Ports, TCP and UDP

- **Port** = a numbered door on a computer for a specific service (0–65535). **Socket** = IP + port, e.g. `10.0.1.5:443`.

	TCP	UDP
Connection	Yes (3-way handshake)	No
Reliability	Guaranteed, ordered, retransmits	Best effort
Speed	Slower	Faster
Used for	Web, SSH, databases, email	DNS, video calls, streaming, gaming

TCP 3-way handshake: `SYN → SYN-ACK → ACK`

Port	Service	Port	Service
22	SSH	443	HTTPS
25	SMTP (mail)	3306	MySQL
53	DNS	5432	PostgreSQL
80	HTTP	6379	Redis
123	NTP (time)	8080	Common app/Jenkins port
3389	RDP (Windows remote)	9090 / 3000	Prometheus / Grafana

2.9 HTTP and HTTPS

Method	Use	Status code	Meaning
GET	Read data	200	OK
POST	Create data	201	Created
PUT / PATCH	Update data	301 / 302	Redirect
DELETE	Delete data	400	Bad request
		401 / 403	Not logged in / not allowed
		404	Not found
		500	Server error
		502 / 503 / 504	Bad gateway / unavailable / timeout (often a backend or load balancer problem)

- **Headers** carry extra info (Host, Content-Type, Authorization, Cookie).
- **HTTPS** = HTTP encrypted with **TLS**. The server proves its identity with a **certificate** signed by a trusted **Certificate Authority (CA)**.
- **REST API** = a server that accepts HTTP requests and returns data, usually JSON.

2.10 Routing, NAT, firewalls, VPN

Term	Meaning
Routing table	A list of "to reach network X, send to Y". `0.0.0.0/0` = default route (everything else)
Hop	Each router a packet passes through
NAT	Translates private IPs to a public IP so private devices can reach the internet. Your home router does this
Firewall	Allows or blocks traffic by IP, port and protocol
Stateful firewall	Remembers connections; return traffic is allowed automatically (AWS Security Groups)
Stateless firewall	Checks every packet; return traffic needs its own rule (AWS NACLs)
VPN	An encrypted tunnel between networks or from a user to a network

2.11 Load balancers, proxies, CDN

Term	Meaning
Load balancer	Spreads traffic across many servers and sends traffic only to healthy ones (health checks)
L4 load balancer	Works on IP and port (TCP/UDP). Fast. AWS NLB
L7 load balancer	Understands HTTP: routes by path or host. AWS ALB
Forward proxy	Sits in front of **clients** and makes requests for them (company web filter)
Reverse proxy	Sits in front of **servers** and receives requests for them (nginx, ALB)
CDN	Caches content in many locations close to users (CloudFront, Cloudflare)

2.12 SSH

Secure Shell: encrypted remote login to a server on port 22. Uses a **key pair**: the private key stays on your machine and must never be shared; the public key goes on the server.

2.13 Networking commands

Windows	Linux	Purpose
`ipconfig /all`	`ip a`	Show IP and MAC addresses
`ping host`	`ping -c 4 host`	Test reachability (ICMP)
`tracert host`	`traceroute host` / `mtr host`	Show each hop to the destination
`nslookup name`	`dig name` / `dig +short name`	DNS lookup
`route print`	`ip route`	Show the routing table
`arp -a`	`ip neigh`	Show the ARP table
`netstat -ano`	`ss -tulpn`	Show listening ports and connections

Windows	Linux	Purpose
`Test-NetConnection host -Port 443`	`nc -zv host 443`	Test whether a port is open
`curl.exe -v https://site`	`curl -v https://site`	Make an HTTP request and see the details
`ipconfig /flushdns`	`resolvectl flush-caches`	Clear the DNS cache
—	`tcpdump -i any port 53`	Capture packets

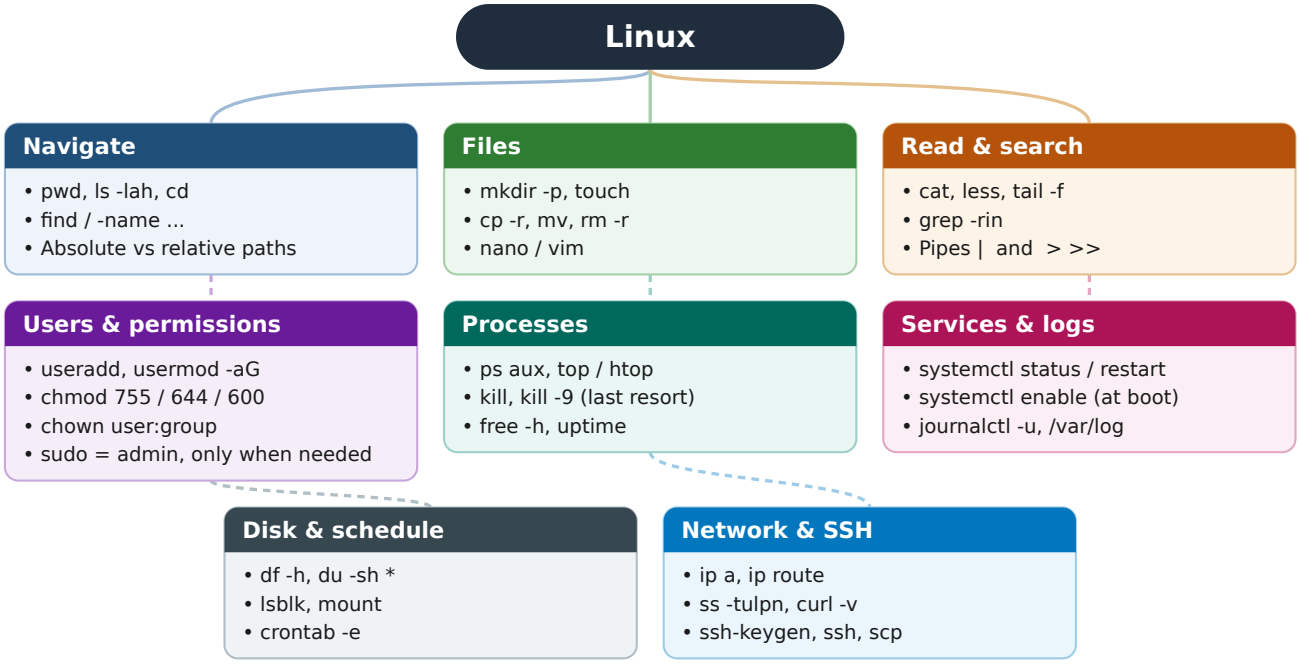
Wireshark = a graphical packet capture tool. Useful filters: `icmp`, `dns`, `tcp.port == 443`, `ip.addr == 10.0.0.5`.

2.14 Network troubleshooting order

1. Is there a link? (Wi-Fi / cable, `ip a`)
2. Do I have a valid IP? (not `169.254.x.x`)
3. Can I reach the gateway? (`ping <gateway>`)
4. Can I reach the internet by IP? (`ping 8.8.8.8`)
5. Does DNS work? (`nslookup google.com`)
6. Is the port open? (`Test-NetConnection` / `nc -zv`)
7. Does the application answer? (`curl -v`)
8. Is a firewall or security group blocking it?

> **REMEMBER**
>
> - OSI layers 7 → 1: **A**ll **P**eople **S**eem **T**o **N**eed **D**ata **P**rocessing.
> - Status codes: **2xx** OK · **3xx** moved · **4xx** you made a mistake · **5xx** the server made a mistake.
> - Troubleshoot: link → IP → gateway → internet (8.8.8.8) → DNS → port → application.
> - Subnet hosts = $2^{\text{host bits}} - 2$. Block size = 256 − mask octet.

3. Linux



3.1 Architecture

```
Applications  (nginx, python, docker)
     |
Shell  (bash: reads your commands)
     |
Kernel  (manages CPU, memory, disks, network, processes)
     |
Hardware
```

- **Distribution (distro):** the kernel plus tools, packaged together. Ubuntu/Debian use `apt`. RHEL/Amazon Linux/Rocky use `dnf` / `yum`.
- **Everything is a file:** devices, settings and processes all appear as files.

3.2 Reading a command

```
ls   -l   -a   /etc
|     └─options─┘   |
command            argument (what to act on)
```

- Options can be combined: `ls -la`. Long form: `--all`.
- Help: `man ls`, `ls --help`. Quit `man` with `q`.
- Shortcuts: `Tab` = autocomplete, `↑` = previous command, `Ctrl+C` = stop, `Ctrl+L` = clear, `history` = past commands.

3.3 File system layout

Directory	Contains
`/`	Root: top of everything
`/home/<user>`	Users' personal files (`~` = your home)
`/root`	Home of the root (admin) user
`/etc`	Configuration files
`/var`	Changing data: logs (`/var/log`), web files, caches
`/bin` , `/usr/bin`	Programs (commands)
`/tmp`	Temporary files, cleared on reboot
`/opt`	Optional / third-party software
`/dev`	Devices (disks, terminals)
`/proc`	Live kernel and process information
`/mnt` , `/media`	Mount points for extra disks

Absolute path: starts from `/` (`/etc/nginx/nginx.conf`). **Relative path:** starts from where you are (`./logs` , `../config`). `.` = here, `..` = one level up.

3.4 Files and directories

Command	What it does
`pwd`	Print the current directory
`ls -lah`	List all files, long format, human-readable sizes
`cd /path` , `cd ~` , `cd ..` , `cd -`	Change directory: path, home, up, previous
`mkdir -p a/b/c`	Create directories (with parents)
`touch file`	Create an empty file / update its timestamp
`cp src dst` , `cp -r dir1 dir2`	Copy a file / a directory
`mv old new`	Move or rename
`rm file` , `rm -r dir`	Delete. **No recycle bin.** Be careful with `rm -rf`
`ln -s target link`	Create a shortcut (symbolic link)
`file name`	Show the file type
`tree`	Show a directory tree

Wildcards: `*` = anything, `?` = one character. Example: `ls *.log` .

3.5 Viewing and searching

Command	What it does
`cat file`	Print the whole file

Command	What it does
`less file`	Scroll through a file (`/word` to search, `q` to quit)
`head -n 20 file` , `tail -n 20 file`	First / last 20 lines
`tail -f /var/log/syslog`	Follow a log live
`grep "error" file`	Find lines containing the text
`grep -i -r -n "error" /var/log`	Case-insensitive, recursive, with line numbers
`grep -v "debug"`	Lines that do **not** match
`find / -name "*.conf"`	Find files by name
`find /var -size +100M`	Find files bigger than 100 MB
`find . -mtime -1`	Files changed in the last day

3.6 Pipes and redirection

Symbol	Meaning	Example
`|`	Send the output of one command into another	`cat log | grep error`
`>`	Write output to a file (overwrite)	`ls > list.txt`
`>>`	Append output to a file	`echo hi >> notes.txt`
`2>`	Redirect errors	`cmd 2> errors.txt`
`&>` / `2>&1`	Redirect output and errors together	`cmd > all.txt 2>&1`
`<`	Use a file as input	`wc -l < file`

Streams: **stdin** (0) = input, **stdout** (1) = normal output, **stderr** (2) = errors.

3.7 Text processing

Command	What it does	Example
`wc -l`	Count lines	`wc -l access.log`
`sort` , `sort -n` , `sort -r`	Sort lines (numeric, reverse)	`sort names.txt`
`uniq -c`	Count repeated lines (input must be sorted)	`sort ips | uniq -c`
`cut -d',' -f2`	Pick a column	`cut -d':' -f1 /etc/passwd`
`awk '{print $1}'`	Print a field	`awk '{print $1}' access.log`
`sed 's/old/new/g'`	Find and replace	`sed -i 's/8080/80/g' app.conf`
`tr` , `xargs`	Translate characters / build commands from input	`cat files | xargs rm`

Top 5 IPs in a web log: `awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -5`

Editors: `nano file` (simple: `Ctrl+O` save, `Ctrl+X` exit). `vim file` (`i` = insert mode, `Esc` = command mode, `:wq` = save and quit, `:q!` = quit without saving).

Archives: `tar -czvf backup.tar.gz dir/` (create), `tar -xzvf backup.tar.gz` (extract), `zip` / `unzip`.

3.8 Users and groups

Command / file	What it does
`whoami`, `id`	Current user, its UID and groups
`sudo command`	Run one command as root (admin)
`sudo useradd -m -s /bin/bash alice`	Create a user with a home directory
`sudo passwd alice`	Set a password
`sudo usermod -aG devops alice`	Add a user to a group (`-a` = append, don't forget it)
`sudo groupadd devops`	Create a group
`sudo userdel -r alice`	Delete a user and their home directory
`su - alice`	Switch to another user
`/etc/passwd`, `/etc/group`, `/etc/shadow`	Users, groups, password hashes
`sudo visudo`	Safely edit sudo permissions

Best practice: never work as root day to day. Use `sudo` only when needed (least privilege).

3.9 Permissions

```
-rwxr-x---  1  alice  devops  4096  app.sh
|└┘└┘└┘       owner  group
| |  |  └ others: ---  (nothing)
| |  └ group:  r-x  (read, execute)
| └ owner:     rwx  (read, write, execute)
└ type: - file, d directory, l link
```

Letter	Number	On a file	On a directory
r	4	Read content	List files
w	2	Change content	Create/delete files inside
x	1	Run as a program	Enter (`cd`) it

Command	Meaning
`chmod 755 file`	rwx r-x r-x (scripts, directories)
`chmod 644 file`	rw- r-- r-- (normal files)
`chmod 600 key.pem`	rw- --- --- (private keys, secrets)
`chmod u+x script.sh`	Add execute for the owner
`chown alice:devops file`	Change owner and group
`chown -R` / `chmod -R`	Apply recursively
`umask`	Default permissions for new files

Special bits: setuid (runs as the file owner, e.g. `passwd`), setgid (new files inherit the group), sticky bit (only the owner can delete, e.g. `/tmp`).

3.10 Processes, CPU and memory

Command	What it does
`ps aux`	All running processes
`ps aux | grep nginx`	Find a process
`top` / `htop`	Live CPU and memory view (`q` to quit; in top, `P` = sort by CPU, `M` = by memory)
`kill PID`	Ask a process to stop (SIGTERM, 15)
`kill -9 PID`	Force-kill (SIGKILL). Last resort
`pkill name`	Kill by name
`command &`, `jobs`, `fg`, `bg`	Run in the background and manage jobs
`nohup command &`	Keep running after you log out
`nice -n 10 cmd`, `renice`	Lower / change priority
`free -h`	RAM and swap usage
`uptime`	Load average for 1, 5 and 15 minutes
`nproc`	Number of CPU cores
`vmstat 1`, `iostat`	CPU, memory and disk statistics

Load average: compare it with the number of cores. Load 4.0 on 4 cores = fully busy. **Swap** = disk used as extra RAM; heavy swapping = slow.

3.11 Package management (Ubuntu)

Command	What it does
`sudo apt update`	Refresh the list of available packages
`sudo apt upgrade`	Install updates
`sudo apt install nginx`	Install a package
`sudo apt remove nginx` / `purge`	Remove (purge also deletes config)
`apt search name`, `apt show name`	Search / show details
`dpkg -l`	List installed packages

RHEL / Amazon Linux equivalent: `sudo dnf install nginx`.

3.12 Services and systemd

systemd starts and manages services (background programs called daemons).

Command	What it does
`systemctl status nginx`	Is it running? Shows recent logs
`sudo systemctl start / stop / restart nginx`	Control the service now
`sudo systemctl reload nginx`	Re-read config without stopping
`sudo systemctl enable nginx`	Start automatically at boot
`sudo systemctl disable nginx`	Don't start at boot
`systemctl list-units --type=service`	List services
`sudo systemctl daemon-reload`	Reload after editing unit files

Simple unit file `/etc/systemd/system/myapp.service`:

```
[Unit]
Description=My App
After=network.target

[Service]
User=appuser
ExecStart=/usr/bin/python3 /opt/myapp/app.py
Restart=on-failure
EnvironmentFile=/etc/myapp/env

[Install]
WantedBy=multi-user.target
```

3.13 Environment variables and shell config

Command	What it does
`env` / `printenv`	List environment variables
`echo $HOME`, `echo $PATH`	Show a variable
`export APP_ENV=dev`	Set a variable for this session and child processes
`unset APP_ENV`	Remove it
`~/.bashrc`	Runs at every new shell: aliases, exports
`source ~/.bashrc`	Reload it now
`alias ll='ls -lah'`	Create a shortcut
`which python3`	Where a command lives (searched via `PATH`)

Security: never hardcode secrets in scripts or code. Load them from environment variables or a secrets manager, and never commit `.env` files.

3.14 Linux networking

Command	What it does
`ip a`	Interfaces and IP addresses
`ip route`	Routing table (default gateway)

Command	What it does
`ss -tulpn`	Listening ports and the program using each one
`ping`, `traceroute`, `mtr`	Reachability and path
`dig name`, `nslookup name`	DNS lookup
`curl -I https://site`	HTTP headers only
`curl -v http://localhost:8080`	Full request/response details
`wget url`	Download a file
`nc -zv host 22`	Test a TCP port
`sudo tcpdump -i eth0 port 80`	Capture packets
`/etc/hosts`, `/etc/resolv.conf`	Local name overrides / DNS servers in use

nginx reverse proxy and load balancer:

```
upstream backend {
    server 127.0.0.1:5001;
    server 127.0.0.1:5002;
}
server {
    listen 80;
    location / {
        proxy_pass http://backend;
    }
}
```

Test the config with `sudo nginx -t`, then `sudo systemctl reload nginx`.

3.15 SSH

Command	What it does
`ssh-keygen -t ed25519 -C "me@laptop"`	Create a key pair (`~/.ssh/id_ed25519` and `.pub`)
`ssh user@host`	Connect
`ssh -i key.pem ubuntu@1.2.3.4`	Connect with a specific key
`ssh-copy-id user@host`	Install your public key on a server
`scp file user@host:/tmp/`	Copy a file to a server
`rsync -avz dir/ user@host:/backup/`	Efficient sync / copy
`ssh -L 8080:localhost:80 user@host`	Tunnel: local port 8080 → server's port 80

`~/.ssh/config` shortcut:

```
Host web1
    HostName 1.2.3.4
    User ubuntu
    IdentityFile ~/.ssh/web1.pem
```

Hardening (`/etc/ssh/sshd_config`): `PasswordAuthentication no`, `PermitRootLogin no`, keys only, private key `chmod 600`, then `sudo systemctl restart ssh`.

3.16 Logs

Command / path	What it does
`/var/log/syslog` (Ubuntu), `/var/log/messages` (RHEL)	General system log
`/var/log/auth.log`	Logins, sudo, SSH attempts
`/var/log/nginx/access.log` , `error.log`	Web server logs
`journalctl -u nginx`	Logs of one service
`journalctl -u nginx -f`	Follow live
`journalctl --since "1 hour ago"`	Recent logs
`journalctl -p err -b`	Errors since the last boot
`dmesg`	Kernel messages (hardware, out-of-memory kills)
`logrotate`	Rotates and compresses old logs so disks don't fill

3.17 Disks and storage

Command	What it does
`df -h`	Free space per filesystem
`df -i`	Free inodes (a disk can be "full" of tiny files)
`du -sh *`	Size of each item in the current directory
`du -h / --max-depth=1 2>/dev/null | sort -h`	Find the biggest top-level directories
`lsblk`	Disks and partitions
`sudo mkfs.ext4 /dev/xvdf`	Create a filesystem (**erases the disk**)
`sudo mount /dev/xvdf /data`	Attach a filesystem to a directory
`/etc/fstab`	Mounts done automatically at boot

Disk full checklist: `df -h` → `du` to find big directories → old logs (`/var/log`), journal (`journalctl --vacuum-size=200M`), caches, old Docker images (`docker system prune`). Deleted but still-open files: `lsof +L1` .

3.18 cron (scheduled jobs)

```
*  *  *  *  *   command
│  │  │  │  └ day of week (0-7, Sun = 0 or 7)
│  │  │  └─── month (1-12)
│  │  └────── day of month (1-31)
│  └───────── hour (0-23)
└──────────── minute (0-59)
```

Example	Runs
`0 2 * * * /opt/backup.sh`	Every day at 02:00
`*/5 * * * * /opt/check.sh`	Every 5 minutes

Example	Runs
`0 9 * * 1-5`	09:00, Monday to Friday

`crontab -e` (edit), `crontab -l` (list). Always use full paths and redirect output: `>> /var/log/backup.log 2>&1`.

3.19 Linux security basics

- Keep the system updated: `sudo apt update && sudo apt upgrade`.
- Firewall (`ufw`): `sudo ufw allow 22/tcp`, `sudo ufw allow 80`, `sudo ufw enable`, `sudo ufw status`.
- SSH keys only, no root login, consider `fail2ban` to block brute-force attempts.
- Least privilege: separate users, minimal sudo, correct permissions.
- Remove unused packages and services. Check listening ports with `ss -tulpn`.
- Review `/var/log/auth.log` for suspicious logins.

3.20 Linux troubleshooting method

1. **What exactly is wrong?** Error message, since when, what changed?
2. **Is the service running?** `systemctl status`
3. **What do the logs say?** `journalctl -u`, `/var/log/...`
4. **Resources:** `top`, `free -h`, `df -h`
5. **Network:** `ss -tulpn`, `curl localhost`, firewall rules
6. **Config and permissions:** `nginx -t`, `ls -l`
7. Fix one thing at a time, then re-test. Write down what you did.

3.21 More essential commands

Command	What it does
`lsof -i :80`	Which process is using port 80
`lsof +L1`	Deleted files still held open (disk "full" but nothing visible)
`hostnamectl`, `timedatectl`	Show/set hostname / time zone and NTP sync
`uname -a`, `cat /etc/os-release`	Kernel version / Linux distribution
`systemctl --failed`	Services that failed
`dmesg -T | grep -i oom`	Processes killed by the out-of-memory (OOM) killer
`watch -n 2 'df -h'`	Re-run a command every 2 seconds
`stat file`	Detailed file info (size, permissions, timestamps)
`sha256sum file`	Checksum to verify a download
`ulimit -n`	Max open files for the shell (a common limit on busy servers)

3.22 Growing a disk (real-world task, e.g. after enlarging an AWS EBS volume)

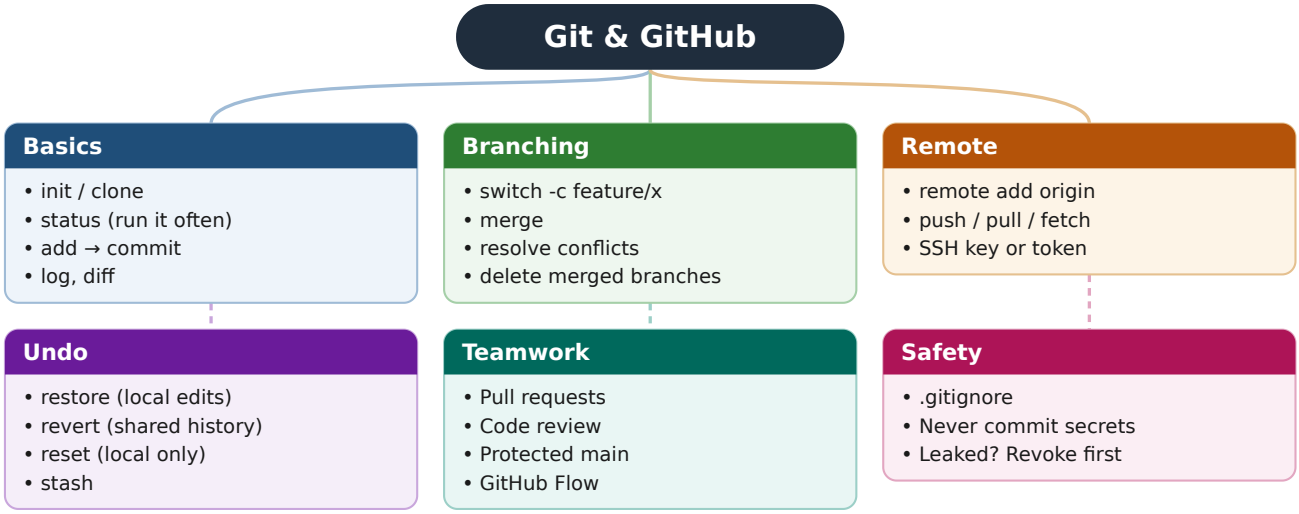
```
lsblk                                  # see the bigger disk and the old partition size
sudo growpart /dev/nvme0n1 1           # grow partition 1 to fill the disk
sudo resize2fs /dev/nvme0n1p1          # grow an ext4 filesystem
sudo xfs_growfs /                      # or: grow an XFS filesystem (Amazon Linux default)
df -h                                  # confirm the new size
```

LVM (Logical Volume Manager): combines disks into flexible volumes. Physical volume (`pvcreate`) → volume group (`vgcreate`) → logical volume (`lvcreate`); grow with `lvextend -r`. **Swap:** extra "memory" on disk. Check with `swapon --show`. Useful on small servers, but heavy swapping means you need more RAM. **SELinux / AppArmor:** extra security layers (RHEL / Ubuntu) that can block a service even when file permissions look right. Check with `getenforce` or `aa-status`.

REMEMBER

- Permissions: **r = 4, w = 2, x = 1** → 7 = rwx, 6 = rw-, 5 = r-x, 4 = r--. Private keys: **600**.
- Cron fields in order: **minute hour day month weekday** ("**M**y **H**amster **D**oesn't **M**ind **W**aiting").
- Service broken? **status → logs → resources → network → config**.
- Linux has no recycle bin: double-check before `rm -rf`.

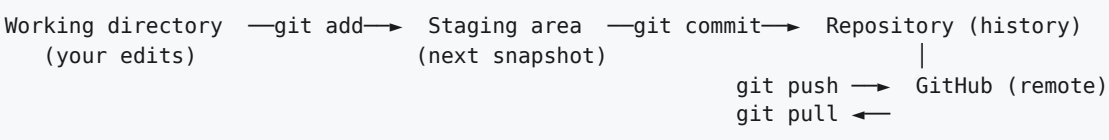
4. Git & GitHub



4.1 Concepts

Term	Meaning
Version control	Records every change so you can see history, undo and work together
Git	Version control tool that runs on your computer
GitHub	A website that hosts Git repositories online, plus pull requests, Actions and more
Repository (repo)	A project folder tracked by Git (the `.git` folder holds its history)
Commit	A saved snapshot with a message, author and unique ID (hash)
Branch	An independent line of work. `main` = the main branch
HEAD	Points to the commit you currently have checked out
Remote	A copy of the repo somewhere else, usually GitHub (`origin`)
Merge	Combine one branch into another
Pull request (PR)	A request to merge a branch, with review and checks
Merge conflict	Both sides changed the same lines; a person must decide

4.2 The three areas



4.3 Everyday commands

Command	What it does
`git config --global user.name "Name"`	Set your name (once)
`git config --global user.email "you@mail.com"`	Set your email (once)
`git init`	Start tracking a folder
`git clone <url>`	Copy a remote repo to your computer
`git status`	What changed / what is staged. Run it often
`git add file` / `git add .`	Stage changes
`git commit -m "Add login page"`	Save a snapshot
`git log --oneline --graph`	Compact history
`git diff` / `git diff --staged`	Unstaged / staged changes
`git show <hash>`	Show one commit

4.4 Branches and merging

Command	What it does
`git branch`	List branches
`git switch -c feature/login`	Create and switch to a new branch
`git switch main`	Switch branch
`git merge feature/login`	Merge that branch into the current one
`git branch -d feature/login`	Delete a merged branch

Resolving a conflict: open the file, find the markers, keep the right code, delete the markers, then `git add` and `git commit`.

```
<<<<<<< HEAD
your version
=======
their version
>>>>>>> feature/login
```

4.5 Remotes

Command	What it does
`git remote -v`	Show remotes
`git remote add origin <url>`	Connect a local repo to GitHub
`git push -u origin main`	Upload and set upstream (first time)
`git push`	Upload commits
`git pull`	Download and merge remote changes
`git fetch`	Download without merging

GitHub authentication: SSH key (`ssh-keygen` , add the `.pub` to GitHub → Settings → SSH keys) or a personal access token. Test with `ssh -T git@github.com` .

4.6 Undoing safely

Situation	Command
Discard unstaged changes in a file	`git restore file`
Unstage a file	`git restore --staged file`
Fix the last commit message (not pushed yet)	`git commit --amend`
Undo a pushed commit safely (adds a new commit)	`git revert <hash>`
Move the branch back (local only, rewrites history)	`git reset --soft/--hard <hash>`
Save unfinished work temporarily	`git stash` , then `git stash pop`

Rule: never rewrite history (`reset` , `--force` push) on shared branches. Use `revert` .

4.7 .gitignore and secrets

```
# .gitignore
.env
*.pem
*.key
node_modules/
__pycache__/
.terraform/
*.tfstate
*.log
```

If a secret is committed: 1) revoke/rotate it immediately (it is compromised), 2) remove it from the code, 3) clean the history if needed, 4) add it to `.gitignore` , 5) use secret scanning (gitleaks, GitHub secret scanning).

4.8 Workflows

Workflow	How it works	Used by
GitHub Flow	`main` is always deployable; short feature branches → PR → merge	Most teams, simple
Trunk-based	Very short branches, merge to `main` many times a day, feature flags	High-speed teams with strong CI
GitFlow	`main` + `develop` + feature/release/hotfix branches	Scheduled releases, older style

Good commits: small, one purpose, clear message in the imperative mood ("Add health check endpoint"). Conventional style: `feat:` , `fix:` , `docs:` , `chore:` . **Tags:** `git tag v1.0.0` + `git push --tags` mark releases. **Protected branches:** require PR review and passing checks before merging into `main` .

4.9 More Git you will use at work

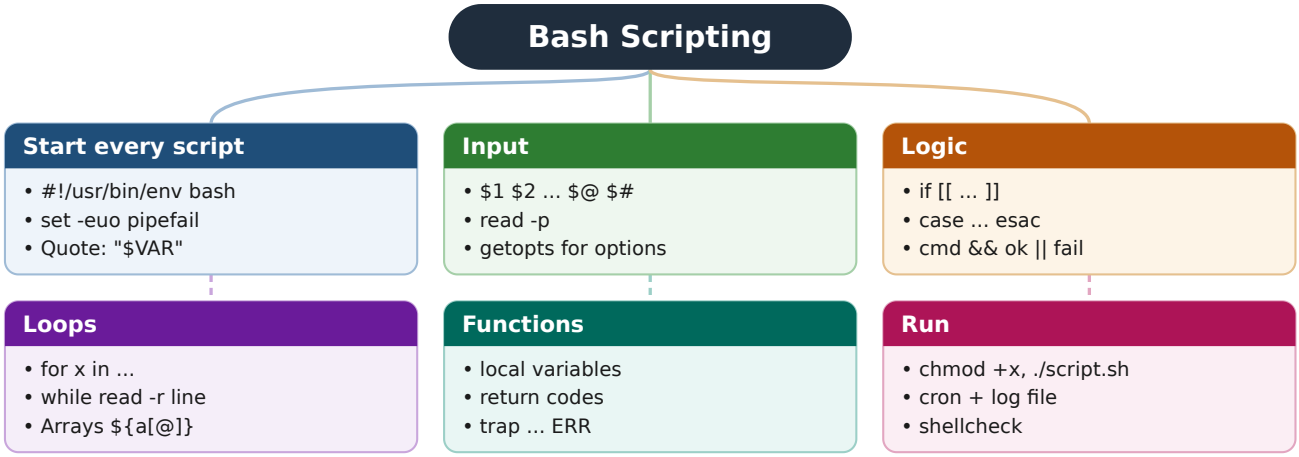
Command	What it does
`git rebase main`	Replay your branch's commits on top of the latest `main` (linear history). Only on your own, unshared branches
`git pull --rebase`	Update your branch without creating merge commits
`git cherry-pick <hash>`	Copy one commit onto the current branch (e.g. a hotfix)
`git blame file`	Who last changed each line, and in which commit
`git log -p file` / `git log --author=name`	History of a file / commits by an author
`git diff main...feature`	What a branch changes compared with `main`
`git tag -a v1.2.0 -m "Release 1.2.0"`	Annotated release tag
`git clean -fd`	Delete untracked files (careful)

Merge vs rebase: merge keeps the real history and adds a merge commit; rebase rewrites your commits into a straight line. Teams choose one style; **never rebase shared branches**. **Semantic versioning:** `MAJOR.MINOR.PATCH` → breaking change . new feature . bug fix (e.g. 2.4.1).

REMEMBER

- The flow: **edit → add → commit → push** (working dir → staging → repo → GitHub).
- Shared branch? Use **revert**, never reset or force-push.
- Small commits, clear messages: "Add health check endpoint".

5. Bash Scripting



5.1 Script basics

```
#!/usr/bin/env bash        # shebang: which program runs the script
set -euo pipefail          # stop on errors, unset variables, and failed pipes

NAME="server1"             # variable (no spaces around =)
echo "Checking $NAME"
echo "Today: $(date +%F)"  # command substitution
```

Run it: `chmod +x script.sh` then `./script.sh`.

Concept	Syntax
Variable	`VAR=value`, use as `"$VAR"` (always quote)
Arguments	`$1`, `$2` ... ; `$#` = count ; `$@` = all ; `$0` = script name
Exit code of last command	`$?` (0 = success, anything else = failure)
Read input	`read -p "Name: " NAME`
Single vs double quotes	`'$VAR'` = literal text, `"$VAR"` = value
Arithmetic	`$((a + b))`

5.2 Conditions

```
if [[ -f "/etc/nginx/nginx.conf" ]]; then
  echo "Config exists"
elif [[ "$ENV" == "prod" ]]; then
  echo "Production"
else
  echo "Missing"
fi
```

Test	True when	Test	True when
`-f file`	File exists	`-z "$s"`	String is empty

Test	True when	Test	True when
`-d dir`	Directory exists	`-n "$s"`	String is not empty
`-x file`	File is executable	`"$a" == "$b"`	Strings are equal
`$a -eq $b`	Numbers equal	`-gt`, `-lt`, `-ge`, `-le`	Greater / less than

Short form: `cmd && echo ok || echo failed`.

5.3 Loops and case

```
for server in web1 web2 web3; do
  ping -c 1 "$server" > /dev/null && echo "$server UP" || echo "$server DOWN"
done

while read -r line; do
  echo "Line: $line"
done < servers.txt

case "$1" in
  start) echo "Starting" ;;
  stop)  echo "Stopping" ;;
  *)     echo "Usage: $0 {start|stop}"; exit 1 ;;
esac
```

5.4 Functions, error handling, logging

```
log() { echo "$(date '+%F %T') [$1] $2"; }

check_service() {
  local svc="$1"
  if systemctl is-active --quiet "$svc"; then
    log INFO "$svc is running"
  else
    log ERROR "$svc is down"
    return 1
  fi
}

trap 'log ERROR "Script failed on line $LINENO"' ERR
check_service nginx
```

5.5 Example: disk usage alert

```
#!/usr/bin/env bash
set -euo pipefail
THRESHOLD=80
USAGE=$(df / --output=pcent | tail -1 | tr -dc '0-9')
if (( USAGE > THRESHOLD )); then
  echo "WARNING: root disk at ${USAGE}%"
  exit 1
fi
echo "OK: root disk at ${USAGE}%"
```

Schedule it with cron: `*/10 * * * * /opt/scripts/disk_check.sh >> /var/log/disk_check.log 2>&1`.

Best practices: quote variables, use `set -euo pipefail`, check inputs, log clearly, use exit codes, check scripts with `shellcheck`, no secrets inside scripts.

5.6 More Bash features

```
servers=("web1" "web2" "db1")            # array
echo "${servers[0]}"                     # first item
echo "${#servers[@]}"                    # number of items
for s in "${servers[@]}"; do echo "$s"; done

name="${1:-default}"                     # default value if $1 is empty
file="report.tar.gz"; echo "${file%.tar.gz}"   # strip a suffix → report

cat <<EOF > /tmp/app.conf                # here-document: write multi-line text
port=8080
env=dev
EOF

while getopts "e:v" opt; do              # parse options: -e <env> -v
  case $opt in
    e) ENV="$OPTARG" ;;
    v) VERBOSE=1 ;;
    *) echo "Usage: $0 -e env [-v]"; exit 1 ;;
  esac
done
```

5.7 Bash syntax: what is what

Annotated script (every line explained in the comments):

```
#!/usr/bin/env bash
# ^ SHEBANG: first line, tells Linux to run this file with bash
# Lines starting with # are COMMENTS (ignored)

set -euo pipefail
# -e  stop if any command fails
# -u  stop if an unset variable is used
# -o pipefail  a pipeline fails if any part of it fails

APP_NAME="web"                # VARIABLE: NAME=value  (no spaces around =)
MAX_RETRIES=3                 # numbers are stored as text, too
LOG_FILE="/tmp/${APP_NAME}.log"  # ${VAR} = use the value, braces mark where the name ends

TODAY=$(date +%F)             # COMMAND SUBSTITUTION: $(command) = its output
COUNT=$(( MAX_RETRIES * 2 ))  # ARITHMETIC: $(( ... ))

log() {                       # FUNCTION definition: name() { ... }
  local level="$1"            # local = variable only inside the function; $1 = first argument
  shift                       # drop $1, so "$*" is the rest of the arguments
  echo "$(date '+%T') [$level] $*" | tee -a "$LOG_FILE"   # | PIPE output into tee; -a = append
}

if [[ $# -lt 1 ]]; then       # IF: [[ test ]]; $# = number of script arguments
  echo "Usage: $0 <url>" >&2  # $0 = script name; >&2 = print to stderr (errors)
  exit 1                      # EXIT CODE: 0 = success, non-zero = failure
fi

URL="$1"
for i in $(seq 1 "$MAX_RETRIES"); do          # FOR loop over 1..3
  # the command itself is the test; > /dev/null = discard its output
  if curl -fsS --max-time 5 "$URL" > /dev/null; then
    log INFO "$URL is up (attempt $i, $TODAY)"
    exit 0
  fi
  log WARN "attempt $i failed, retrying"
  sleep 2
done

log ERROR "$URL is down after $MAX_RETRIES attempts (counter $COUNT)"
exit 2
```

Element	Syntax	Meaning
Variable	`NAME=value` · use `"$NAME"`	No spaces around `=`. Always quote when using
Single quotes	`'$HOME'`	Literal text, nothing expanded
Double quotes	`"$HOME"`	Variables expanded, spaces kept safe
Command substitution	`$(cmd)`	Replaced by the command's output
Arithmetic	`$(( a + b ))`	Integer math: `+ - * / %`
Special variables	`$0` `$1..$9` `$#` `$@` `$?` `$$`	Script name, arguments, count, all args, last exit code, PID
Test	`[[ ... ]]`	Conditions (file, string, number)
Command as condition	`if grep -q x file; then`	Uses the command's exit code
And / or	`cmd1 && cmd2` · `cmd1 || cmd2`	Run cmd2 only if cmd1 succeeds / fails
Redirect	`>` `>>` `2>` `&>` `< file`	Overwrite, append, errors, both, input

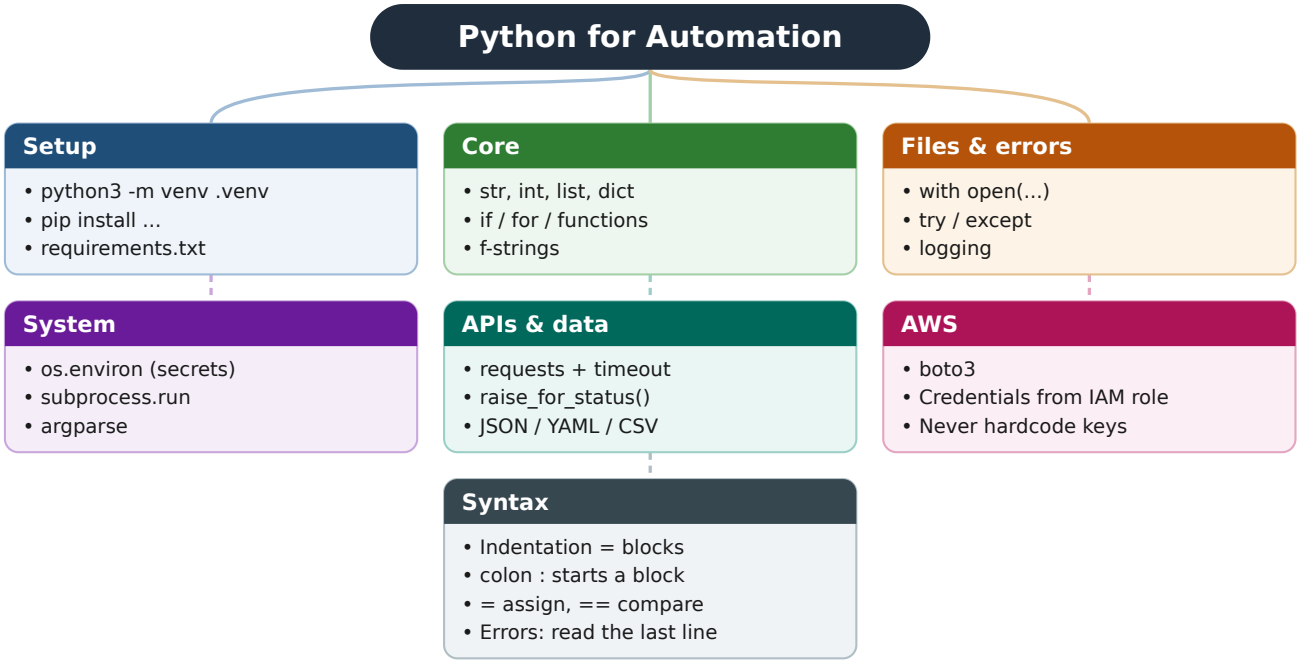
Element	Syntax	Meaning
Pipe	`cmd1 | cmd2`	Output of cmd1 becomes input of cmd2
Discard	`> /dev/null 2>&1`	Throw away output and errors
Function	`name() { ...; }`	Call as `name arg1 arg2`; arguments are `$1`, `$2` inside
Block endings	`if…fi` · `case…esac` · `for/while… do…done`	Bash closes blocks with reversed words
Background	`cmd &`	Run without waiting

Common mistakes: spaces around `=` (`A = 1` fails) · unquoted variables with spaces · missing `then`/`do` · forgetting `chmod +x` · Windows line endings (`\r`) in scripts edited on Windows (fix: `sed -i 's/\r$//' script.sh`).

REMEMBER

- Exit code **0 = success**, anything else = failure (`$?`).
- Tests: `-f` file · `-d` directory · `-z` empty · `-eq/-gt/-lt` numbers · `==` strings.
- Never put passwords in scripts: read them from the environment.

6. Python for Automation



6.1 Setup

Command	What it does
`python3 --version`	Check the version
`python3 -m venv .venv`	Create a virtual environment (isolated packages per project)
`source .venv/bin/activate`	Activate it (`deactivate` to leave)
`pip install requests boto3`	Install packages
`pip freeze > requirements.txt`	Save dependencies
`pip install -r requirements.txt`	Install saved dependencies

6.2 Basics

```
name = "web1"                       # string
port = 8080                         # integer
servers = ["web1", "web2"]          # list
config = {"env": "dev", "port": 80}  # dictionary (key → value)

for s in servers:
    if s == "web1":
        print(f"{s} is primary")   # f-string
    else:
        print(s)

def is_port_valid(p: int) -> bool:
    return 1 <= p <= 65535
```

6.3 Files, errors, environment variables

```
import os, logging, subprocess

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")

api_token = os.environ.get("API_TOKEN")      # secret comes from the environment
if not api_token:
    raise SystemExit("API_TOKEN is not set")

try:
    with open("/var/log/app.log") as f:      # 'with' closes the file automatically
        errors = [line for line in f if "ERROR" in line]
    logging.info("Found %d errors", len(errors))
except FileNotFoundError:
    logging.error("Log file not found")

result = subprocess.run(["df", "-h"], capture_output=True, text=True, check=True)
print(result.stdout)
```

6.4 APIs and JSON

```
import json, requests

resp = requests.get("https://api.github.com/repos/kubernetes/kubernetes", timeout=10)
resp.raise_for_status()                      # error on 4xx/5xx
data = resp.json()                           # JSON → Python dictionary
print(data["stargazers_count"])

with open("report.json", "w") as f:
    json.dump({"stars": data["stargazers_count"]}, f, indent=2)
```

6.5 AWS with boto3

```
import boto3   # uses credentials from AWS CLI config / IAM role, never hardcoded

ec2 = boto3.client("ec2", region_name="us-east-1")
for r in ec2.describe_instances()["Reservations"]:
    for i in r["Instances"]:
        print(i["InstanceId"], i["State"]["Name"])
```

Good habits: virtual environments, `requirements.txt`, `timeout=` on network calls, `try/except` with clear messages, logging instead of print in real tools, secrets from environment or a secrets manager.

6.6 Structure of a real automation script

```
#!/usr/bin/env python3
# Check that a list of URLs answers with HTTP 200.
import argparse, logging, sys
import requests

def check(url: str) -> bool:
    try:
        return requests.get(url, timeout=5).status_code == 200
    except requests.RequestException as e:
        logging.error("%s failed: %s", url, e)
        return False

def main() -> int:
    parser = argparse.ArgumentParser(description="URL health checker")
    parser.add_argument("urls", nargs="+", help="URLs to check")
    args = parser.parse_args()
    logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
    results = {u: check(u) for u in args.urls}
    for u, ok in results.items():
        logging.info("%s %s", "UP  " if ok else "DOWN", u)
    return 0 if all(results.values()) else 1        # exit code for CI/cron

if __name__ == "__main__":                          # run only when executed directly
    sys.exit(main())
```

Run: `python3 check.py https://example.com https://api.example.com`. Data formats you will parse daily: **JSON** (`json` module), **YAML** (`pip install pyyaml` → `yaml.safe_load`), **CSV** (`csv` module).

6.7 Python basics: what is what

Data types

Type	Example	What it is
`str` (string)	`"web1"` or `'web1'`	Text, always in quotes
`int`	`8080`	Whole number
`float`	`99.5`	Decimal number
`bool`	`True` / `False`	Yes/no (capital first letter)
`None`	`None`	"No value"
`list`	`["web1", "web2"]`	Ordered, changeable collection; index starts at 0
`tuple`	`("10.0.0.1", 22)`	Ordered, **unchangeable** collection
`dict`	`{"name": "web1", "port": 80}`	Key → value pairs
`set`	`{"dev", "prod"}`	Unique items, no order

Check a type with `type(x)`. Convert with `int("80")`, `str(80)`, `float("1.5")`, `list(...)`.

Strings

```
name = "web-server"
env = 'prod'
multi = """line one
line two"""                        # triple quotes = multi-line string

print(f"{name} runs in {env}")   # f-string: put variables inside {}
print(name.upper())              # WEB-SERVER
print("  text  ".strip())        # "text"  (remove spaces at both ends)
print(name.replace("-", "_"))    # web_server
print(name.split("-"))           # ['web', 'server']   (string → list)
print(",".join(["a", "b"]))      # a,b                 (list → string)
print(name.startswith("web"))    # True
print("server" in name)          # True  (contains?)
print(len(name))                 # 10    (length)
print(name[0], name[-1], name[0:3])   # w r web   (index and slicing)
```

Numbers and operators

Operators	Meaning	Example
`+ - * /`	Math (`/` always gives a float)	`7 / 2` → `3.5`
`// % **`	Whole division, remainder, power	`7 // 2` → 3, `7 % 2` → 1, `2 ** 3` → 8
`== != > < >= <=`	Comparison → True/False	`cpu > 80`
`and or not`	Logic	`cpu > 80 and env == "prod"`
`in`, `not in`	Membership	`"web1" in servers`
`=` vs `==`	Assign vs compare	`x = 5` sets, `x == 5` tests

Lists and dictionaries

```
servers = ["web1", "web2"]
servers.append("web3")           # add to the end
servers.remove("web1")           # remove by value
print(servers[0], len(servers))  # web2 2
for s in servers:                # loop over items
    print(s)

server = {"name": "web1", "port": 80}
print(server["name"])            # web1  (KeyError if missing)
print(server.get("env", "dev"))  # dev   (safe: default if missing)
server["env"] = "prod"           # add / update a key
for key, value in server.items():
    print(key, value)

ports = [p * 10 for p in range(1, 4)]   # list comprehension → [10, 20, 30]
```

Control flow and functions

```
cpu = 85
if cpu > 90:                      # colon : starts a block
    status = "critical"           # INDENTATION (4 spaces) defines the block
elif cpu > 70:
    status = "warning"
else:
    status = "ok"

for i in range(3):                # 0, 1, 2
    if i == 1:
        continue                  # skip to the next loop
    print(i)

attempts = 0
while attempts < 3:               # repeat while the condition is True
    attempts += 1                 # same as attempts = attempts + 1

def check_port(host: str, port: int = 22) -> bool:   # parameters, default value, type hints
    """Docstring: explains what the function does."""
    return port in (22, 80, 443)                      # return sends a value back

print(check_port("web1"), check_port("web1", 8080))   # True False
```

Modules, input and output

```
import os                         # standard library module
from pathlib import Path          # import one thing from a module

home = os.environ.get("HOME")     # read an environment variable
path = Path("/tmp") / "report.txt"
path.write_text("hello\n")        # write a file
print(path.read_text())           # read a file
```

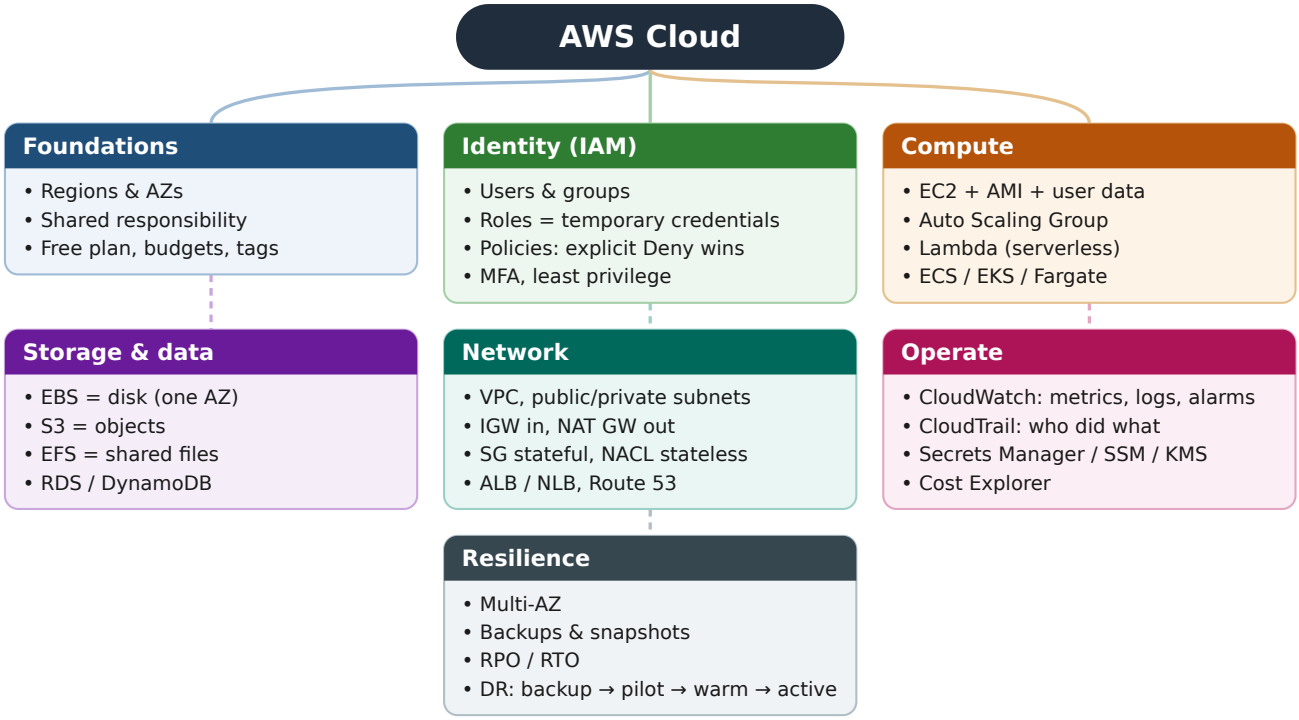
Common errors and what they mean

Error	Meaning
`IndentationError`	Spaces/tabs wrong in a block
`SyntaxError`	Missing `:`, bracket or quote
`NameError`	Variable or function not defined (typo?)
`TypeError`	Wrong type, e.g. `"port" + 80` (use `f"port {80}"`)
`KeyError` / `IndexError`	Missing dict key / list index out of range
`ModuleNotFoundError`	Package not installed in this venv (`pip install ...`)
`FileNotFoundError`	Wrong path

REMEMBER

- Three habits: **virtual environment**, **timeout on every network call**, **secrets from the environment**.
- Return a non-zero exit code on failure so cron/CI notices.

7. AWS Cloud



7.1 Cloud concepts

Term	Meaning
Cloud computing	Renting IT resources on demand, paying for what you use
IaaS	You get raw servers, networks and disks (EC2, VPC)
PaaS	The platform runs your code; you don't manage servers (Elastic Beanstalk, RDS)
SaaS	Finished software you just use (Gmail, Slack)
Region	A geographic area with its own data centers (`us-east-1` , `ap-south-1` Mumbai)
Availability Zone (AZ)	One or more separate data centers inside a region (`ap-south-1a`). Use 2+ for high availability
Edge location	Small sites close to users, used by CloudFront and Route 53
Shared responsibility	AWS secures the cloud (hardware, buildings); **you** secure what's in it (data, IAM, OS patches, firewall rules)
Elasticity / scalability	Grow and shrink resources with demand
High availability (HA)	The system keeps working when one part fails

Pricing models: On-Demand (pay per second/hour), Savings Plans / Reserved (1–3 year commitment, cheaper), Spot (spare capacity, up to ~90% cheaper, can be taken back). **Free plan (new accounts since July 2025):** $100 credit at sign-up, plus up to $100 for completing activities; lasts 6 months or until the credits are used.

7.2 Account safety first

1. Enable **MFA on the root user**, then stop using root.
2. Create an admin identity (IAM Identity Center user, or an IAM user with MFA).
3. Create a **budget** with email alerts (Billing → Budgets).
4. Choose one region and stay in it.
5. **Tag** everything: `Project`, `Owner`, `Env`.
6. After every lab: delete resources and check the Billing console.

7.3 IAM (Identity and Access Management)

Term	Meaning
User	A person or app with long-term credentials
Group	A set of users that share permissions
Role	A set of permissions that is **assumed** temporarily (by EC2, Lambda, CI pipelines, other accounts). No long-term keys
Policy	A JSON document that allows or denies actions
STS	Security Token Service: issues temporary credentials
IAM Identity Center	Central login (SSO) for people, giving short-lived credentials
Least privilege	Give only the permissions that are actually needed

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:ListBucket"],
    "Resource": ["arn:aws:s3:::my-bucket", "arn:aws:s3:::my-bucket/*"]
  }]
}
```

Evaluation rule: everything is denied by default → an explicit Allow grants → an explicit **Deny always wins**. **Best practices:** MFA everywhere, roles instead of access keys, no keys in code, rotate or delete unused keys, use IAM Access Analyzer.

7.4 AWS CLI

Command	What it does
`aws configure sso`	Log in through IAM Identity Center (preferred)
`aws configure`	Set up access keys (avoid if possible)
`aws sts get-caller-identity`	"Who am I?" — always check this first
`aws s3 ls`, `aws s3 cp file s3://bucket/`	List buckets / upload
`aws ec2 describe-instances --output table`	List instances
`--profile dev`, `--region ap-south-1`	Choose a profile / region
`--query 'Reservations[].Instances[].InstanceId'`	Filter output (JMESPath)

7.5 EC2 (virtual servers)

Term	Meaning
Instance	A virtual server
AMI	The image the server starts from (Ubuntu, Amazon Linux)
Instance type	Size: `t3.micro` (2 vCPU, 1 GB), `m5.large`. Families: t = burstable, m = general, c = compute, r = memory
Key pair	SSH keys to log in (download the `.pem` once, `chmod 400`)
User data	A script that runs at first boot (install software automatically)
Instance metadata	Info the instance can read about itself at `169.254.169.254` (use IMDSv2)
Instance profile	Attaches an IAM role to EC2, so apps on it get credentials without keys

States: pending → running → stopping → stopped (no compute charge, disks still charged) → terminated (deleted).

```
#!/bin/bash
# user data example
dnf install -y nginx
systemctl enable --now nginx
```

7.6 Storage: EBS, instance store, EFS, S3

Service	Type	Use
EBS	Block disk attached to one EC2 in one AZ	OS disk, databases. Types: gp3 (general), io2 (high IOPS)
Snapshot	Point-in-time backup of an EBS volume, stored in S3	Backups, copying disks to other AZs/regions
Instance store	Temporary disk on the host. **Lost on stop**	Cache, scratch space
EFS	Shared network file system for many instances	Shared files
S3	Object storage, unlimited	Files, backups, static websites, logs

7.7 Security Groups and Elastic IP

- **Security Group (SG):** a stateful firewall around an instance. Allow rules only. Return traffic is allowed automatically. Can reference other SGs (e.g. "allow 5432 from the app SG").
- **Good practice:** SSH (22) only from your IP (or use SSM Session Manager and close 22), HTTP/HTTPS from `0.0.0.0/0` only on the load balancer.
- **Elastic IP:** a static public IPv4 address. All public IPv4 addresses are billed per hour, so release unused ones.

7.8 S3

Term	Meaning
Bucket	A container with a globally unique name
Object / key	A file and its full name (`logs/2026/app.log`)
Storage classes	Standard, Standard-IA, Intelligent-Tiering, Glacier (cheaper, slower to retrieve)
Versioning	Keeps old versions; protects against deletes and overwrites
Lifecycle rules	Move to cheaper classes or delete after N days
Encryption	SSE-S3 (default) or SSE-KMS
Block Public Access	Keep ON unless you truly need a public bucket
Bucket policy	JSON permissions on the bucket
Presigned URL	A temporary link to a private object

7.9 VPC (your private network)

```
VPC 10.0.0.0/16  (one region)
├── AZ-a
│   ├── Public subnet  10.0.1.0/24   → route 0.0.0.0/0 → Internet Gateway   [ALB, NAT GW, bastion]
│   └── Private subnet 10.0.11.0/24  → route 0.0.0.0/0 → NAT Gateway        [app servers]
├── AZ-b
│   ├── Public subnet  10.0.2.0/24
│   └── Private subnet 10.0.12.0/24                                          [RDS standby]
└── Internet Gateway
```

Component	Meaning
VPC	An isolated network in one region, with your chosen CIDR
Subnet	A range inside the VPC, in exactly one AZ
Public subnet	Its route table has a route to an **Internet Gateway**
Private subnet	No direct route from the internet
Route table	Rules for where traffic goes. `local` route = inside the VPC
Internet Gateway (IGW)	Connects the VPC to the internet (both directions)
NAT Gateway	Lets private instances reach the internet (outbound only). Sits in a public subnet. **Billed hourly + per GB: delete after labs**
NACL	Stateless firewall on the subnet. Allow and deny rules, numbered order. Return traffic needs rules (ephemeral ports 1024–65535)
VPC endpoint	Private access to AWS services (S3, ECR) without going through the internet/NAT
VPC peering	Connect two VPCs privately (CIDRs must not overlap; not transitive)
Transit Gateway	A hub that connects many VPCs and on-premises networks
Site-to-Site VPN / Direct Connect	Encrypted tunnel / private line to an office or data center
VPC Flow Logs	Records of accepted/rejected traffic: for troubleshooting and security
Reachability Analyzer	Tells you why A cannot reach B

	Security Group	NACL
Level	Instance (network interface)	Subnet
State	Stateful	Stateless
Rules	Allow only	Allow and deny
Evaluation	All rules together	In number order, first match wins

SSM Session Manager: a shell on an instance through AWS, with no SSH port open and no keys. Needs the SSM agent + an IAM role.

7.10 Load balancing and Auto Scaling

Term	Meaning
ALB	Layer 7 (HTTP/HTTPS): path/host routing, TLS termination
NLB	Layer 4 (TCP/UDP): very fast, static IPs
Listener	Port + protocol the load balancer listens on (e.g. 443)
Target group	The servers (or containers) traffic is sent to, with a health check
Health check	Regular request (e.g. `GET /health`); unhealthy targets get no traffic
Launch template	The recipe for new instances: AMI, type, SG, user data, role
Auto Scaling Group (ASG)	Keeps min/desired/max instances, replaces unhealthy ones, scales on metrics
Target tracking policy	"Keep average CPU at 50%"

HA web tier: ALB across 2 AZs → ASG across 2 private subnets → RDS Multi-AZ.

7.11 Route 53 and ACM

Term	Meaning
Hosted zone	The DNS records for a domain
Alias record	AWS-specific record pointing to AWS resources (ALB, CloudFront, S3). Free and works at the domain root
Routing policies	Simple, Weighted (A/B), Latency, Failover (with health checks), Geolocation
ACM	Free public TLS certificates for ALB/CloudFront, renewed automatically

7.12 RDS (managed databases)

Term	Meaning
RDS	Managed MySQL, PostgreSQL, MariaDB, SQL Server, Oracle. AWS handles patching and backups
Aurora	AWS's high-performance MySQL/PostgreSQL-compatible engine
Multi-AZ	A standby copy in another AZ with automatic failover (**availability**)
Read replica	A read-only copy for scaling reads (**performance**)

Term	Meaning
Automated backups / snapshots	Point-in-time restore / manual backups
DB subnet group	The private subnets the database may use

Security: private subnets only, SG allows the DB port only from the app's SG, encryption on, password in Secrets Manager.

7.13 Monitoring and audit

Service	Use
CloudWatch Metrics	CPU, network, disk, custom metrics
CloudWatch Alarms	Trigger on a threshold → SNS email, Auto Scaling, etc.
CloudWatch Logs + Logs Insights	Central logs and log search
CloudWatch Agent	Sends memory/disk metrics and log files from instances (not collected by default)
CloudTrail	Audit log of **who did what** via API/console. Keep it enabled
EventBridge	Reacts to events or schedules (e.g. run a Lambda every night)
SNS	Notifications (email, SMS, other services)
AWS Config	Records resource configuration and checks rules

7.14 Secrets and encryption

Service	Use
Secrets Manager	Stores secrets (DB passwords, API keys), supports automatic rotation. Paid per secret
SSM Parameter Store	Stores config and secrets (SecureString). Standard tier is free
KMS	Creates and manages encryption keys used by S3, EBS, RDS, Secrets Manager

Apps read secrets at runtime using their IAM role. **Never** put secrets in code, AMIs, user data or Git.

7.15 Serverless and containers on AWS

Service	Use
Lambda	Run code without servers, triggered by events; pay per request and duration (max 15 min)
API Gateway	HTTP endpoints in front of Lambda or other backends
ECR	Private container image registry
ECS	AWS container orchestrator: task definitions → services
Fargate	Run containers without managing servers (works with ECS and EKS)
EKS	Managed Kubernetes. Control plane billed hourly + worker nodes

7.16 Cost management

- Budgets + alerts, Cost Explorer, cost allocation **tags**, Free plan credit balance.
- Common surprise bills: NAT Gateways, idle load balancers, unattached EBS volumes and old snapshots, public IPv4 addresses, EKS clusters, large data transfer, forgotten resources in another region.
- Save money: right-size instances, stop dev servers at night, Savings Plans/Spot, S3 lifecycle rules, delete unused resources.

7.17 Backup and disaster recovery

Term	Meaning
RPO	Recovery Point Objective: how much data you can afford to lose (e.g. 15 minutes)
RTO	Recovery Time Objective: how quickly you must be back online (e.g. 1 hour)
AWS Backup	Central backup plans for EBS, RDS, EFS, DynamoDB...

DR strategy	Cost	Recovery speed
Backup & restore	$	Hours
Pilot light (core parts always running)	$$	Tens of minutes
Warm standby (small full copy running)	$$$	Minutes
Multi-site active/active	$$$$	Near zero

Well-Architected Framework pillars: Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, Sustainability.

7.18 AWS troubleshooting quick checks

Problem	Check in this order
Can't SSH to EC2	Instance running + status checks → public IP → SG allows 22 from your IP → NACL → route table has IGW → correct key and username (`ubuntu` , `ec2-user`) → key permissions `chmod 400`
Private server has no internet	Route `0.0.0.0/0` → NAT GW → NAT GW is in a public subnet with an IGW route → SG/NACL outbound
ALB shows 502/503	Target group health → app listening on the right port → SG allows ALB → targets → health check path returns 200
App can't reach RDS	Same VPC/peering → DB SG allows the port from the app SG → correct endpoint and port → credentials → DB status
Access Denied	`aws sts get-caller-identity` → which policy applies → explicit Deny? → resource policy (bucket policy) → right region/account

7.19 Multi-account setup (how companies really use AWS)

Service	Purpose
AWS Organizations	Groups many accounts (dev, staging, prod, security, logging) under one management account with consolidated billing

Service	Purpose
Service Control Policies (SCPs)	Guardrails for whole accounts, e.g. "nobody may disable CloudTrail" or "only these regions". They limit, they never grant
AWS Control Tower	Sets up a well-governed multi-account landing zone automatically
IAM Identity Center	One login for people across all accounts, with permission sets

Why separate accounts: a mistake or breach in dev can't touch prod, clear cost per team/environment, simpler permissions.

7.20 Security services

Service	What it does
GuardDuty	Threat detection from CloudTrail, VPC Flow Logs and DNS logs (e.g. crypto-mining, stolen keys)
Security Hub	Central security findings and best-practice checks
Inspector	Scans EC2, ECR images and Lambda for vulnerabilities
Macie	Finds sensitive data (personal data) in S3
WAF	Web Application Firewall in front of ALB/CloudFront/API Gateway (SQL injection, bad bots, rate limits)
Shield	DDoS protection (Standard is automatic and free)
AWS Config	Records configuration changes and checks compliance rules
IAM Access Analyzer	Finds resources shared outside your account and unused permissions

IAM policy with a condition (example: require MFA):

```
{
  "Effect": "Deny",
  "Action": "*",
  "Resource": "*",
  "Condition": { "BoolIfExists": { "aws:MultiFactorAuthPresent": "false" } }
}
```

7.21 Other services you must recognize

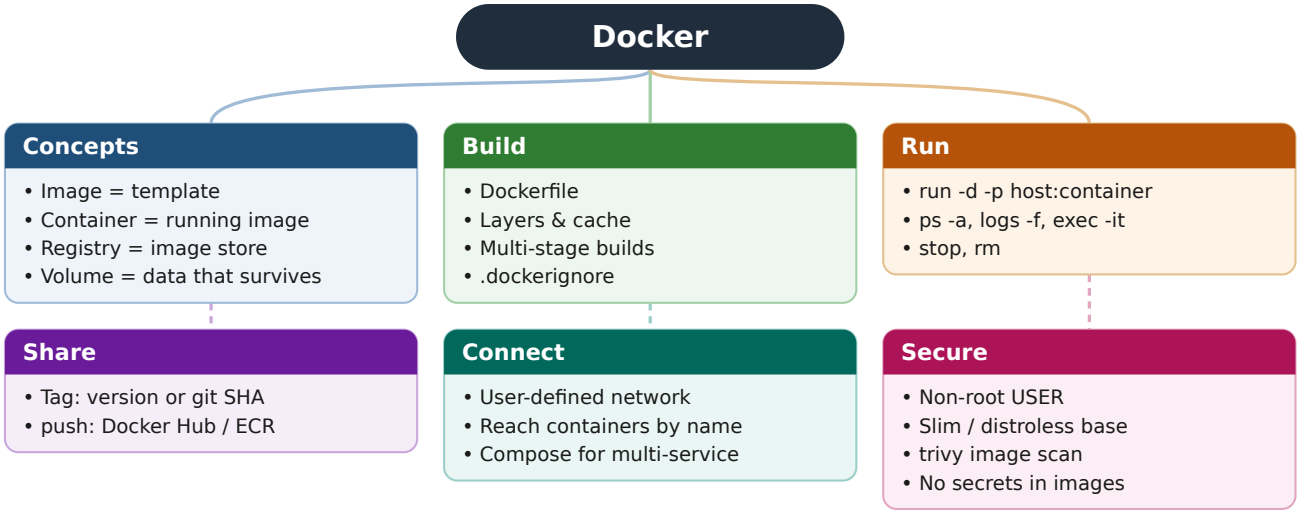
Service	One-line purpose
CloudFront	CDN: caches content close to users, HTTPS, works with S3 and ALB
SQS	Message queue: decouples services (producer → queue → consumer)
SNS	Pub/sub notifications (email, SMS, fan-out to SQS/Lambda)
DynamoDB	Serverless NoSQL key-value database
ElastiCache	Managed Redis/Memcached for caching
EFS	Shared NFS file system
Elastic Beanstalk	Upload code; AWS runs the servers (PaaS)

Service	One-line purpose
Systems Manager (SSM)	Session Manager, Run Command, Patch Manager, Parameter Store
CodeBuild / CodePipeline / CodeDeploy	AWS's own CI/CD services
CloudFormation / CDK	AWS's own infrastructure as code (Terraform alternative)
Trusted Advisor	Checks for cost, security and limit issues
Service Quotas	Account limits (e.g. number of vCPUs) — a common cause of failed launches

REMEMBER

- **Public subnet** = route to an Internet Gateway. Private servers reach the internet through a **NAT Gateway**.
- **Security Group**: stateful, allow-only, per instance. **NACL**: stateless, allow + deny, per subnet.
- First command when anything fails: `aws sts get-caller-identity` (who am I, which account?).
- Well-Architected pillars: Operations, Security, Reliability, Performance, Cost, Sustainability.
- After every lab: **delete resources and check Billing**.

8. Docker



8.1 Concepts

Term	Meaning
Container	An app plus everything it needs, running isolated on a shared OS kernel
Image	A read-only template used to create containers (built from a Dockerfile)
Dockerfile	Text instructions to build an image
Layer	Each instruction creates a cached layer. Unchanged layers are reused
Registry	Stores images: Docker Hub, Amazon ECR, GitHub Container Registry
Tag	Image version label: `myapp:1.2.0`. `latest` is just a default name, not "newest"
Volume	Storage that outlives a container
Docker Engine	The background service (daemon) that builds and runs containers
Namespaces / cgroups	Linux features that isolate containers / limit their CPU and memory

	Virtual machine	Container
Includes	Full guest OS	App + libraries only
Size	GBs	MBs
Start time	Minutes	Seconds
Isolation	Stronger (own kernel)	Shares the host kernel

8.2 Core commands

Command	What it does
`docker version`, `docker info`	Check the installation

Command	What it does
`docker run -d -p 8080:80 --name web nginx`	Run in the background (`-d`), map host 8080 → container 80, name it
`docker run -it ubuntu bash`	Interactive shell in a new container
`docker ps` / `docker ps -a`	Running / all containers
`docker logs -f web`	Follow container logs
`docker exec -it web sh`	Open a shell inside a running container
`docker stop web` , `docker start web` , `docker rm web`	Stop / start / delete
`docker images`	List local images
`docker pull nginx:1.27`	Download an image
`docker rmi image`	Delete an image
`docker inspect web`	Full details (IP, mounts, env) as JSON
`docker stats`	Live CPU/memory per container
`docker system df` , `docker system prune`	Disk usage / clean unused data

8.3 Dockerfile

```
FROM python:3.12-slim              # base image (small "slim" variant)
WORKDIR /app                       # working directory inside the image
COPY requirements.txt .            # copy dependency list first (better caching)
RUN pip install --no-cache-dir -r requirements.txt
COPY . .                           # then copy the code
ENV PORT=8000                      # non-secret default config
EXPOSE 8000                        # documents the port
RUN useradd -m appuser
USER appuser                       # don't run as root
HEALTHCHECK CMD python -c \
  "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"
CMD ["python", "app.py"]           # default command when the container starts
```

Instruction	Purpose
`FROM`	Base image
`RUN`	Run a command at **build** time
`COPY` / `ADD`	Copy files in (prefer COPY)
`WORKDIR`	Set the directory
`ENV` / `ARG`	Runtime variable / build-time variable
`EXPOSE`	Document a port
`USER`	User to run as
`CMD`	Default command (easy to override)
`ENTRYPOINT`	Fixed main command; `CMD` becomes its default arguments

Build and run: `docker build -t myapp:1.0 .` → `docker run -d -p 8000:8000 myapp:1.0`

.dockerignore: `.git`, `.env`, `__pycache__`, `node_modules`, `*.log` (keeps images small and secrets out).

8.4 Multi-stage build

```
FROM golang:1.23 AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -o app

FROM gcr.io/distroless/static     # tiny final image: no shell, no compiler
COPY --from=build /src/app /app
USER nonroot
ENTRYPOINT ["/app"]
```

Build tools stay in the first stage; only the result is shipped. Smaller image = faster and fewer vulnerabilities.

8.5 Registries

Command	What it does
`docker login`	Log in to Docker Hub
`docker tag myapp:1.0 user/myapp:1.0`	Add a registry name to an image
`docker push user/myapp:1.0`	Upload
`aws ecr get-login-password | docker login --username AWS --password-stdin <acct>.dkr.ecr.<region>.amazonaws.com`	Log in to ECR

Tagging: use versions or Git commit SHAs (`myapp:1.4.2`, `myapp:3f9a1c2`), not only `latest`. Turn on immutable tags in ECR.

8.6 Volumes and storage

Type	Example	Use
Named volume	`-v pgdata:/var/lib/postgresql/data`	Database data, managed by Docker
Bind mount	`-v $(pwd)/src:/app`	Live code during development
tmpfs	`--tmpfs /tmp`	Temporary in-memory data

`docker volume ls`, `docker volume inspect pgdata`, `docker volume rm pgdata`. Container filesystems are temporary: data not in a volume is lost when the container is removed.

8.7 Networking

Network	Behaviour
bridge (default)	Private network on the host; publish ports with `-p`
user-defined bridge	Containers can reach each other **by name** (built-in DNS)
host	Container uses the host's network directly
none	No network

```
docker network create appnet
# the password is kept in db.env, not typed in the command
docker run -d --name db  --network appnet --env-file db.env postgres:16
docker run -d --name api --network appnet -p 8000:8000 myapp:1.0   # app connects to host "db"
```

`-p 8080:80` = **host port : container port**. Inside a container, `localhost` means the container itself, not the host.

8.8 Docker Compose

```
# compose.yaml
services:
  web:
    build: .
    ports:
      - "8000:8000"
    env_file: .env              # keep .env out of Git
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: app
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
volumes:
  pgdata:
```

Command	What it does
`docker compose up -d`	Build (if needed) and start everything
`docker compose ps` / `logs -f web`	Status / logs
`docker compose down`	Stop and remove (add `-v` to delete volumes)
`docker compose build`	Rebuild images

8.9 Container security

- Small, trusted, pinned base images (`python:3.12-slim`, distroless).
- Run as non-root (`USER`), read-only filesystem where possible, drop extra Linux capabilities.
- No secrets in images or Dockerfiles. Pass them at runtime (env from a secrets manager, Docker/K8s secrets).
- Scan images: `trivy image myapp:1.0`. Fix HIGH/CRITICAL findings by updating base images and packages.
- Limit resources: `--memory 512m --cpus 1`.

8.10 Container troubleshooting

Symptom	Check
Container exits immediately	`docker ps -a` (exit code) → `docker logs <name>` → is `CMD` correct?
Keeps restarting	`docker logs`, `docker inspect` (RestartCount, OOMKilled), missing env/config
Exit code 137	Killed, usually out of memory (OOMKilled) → raise the limit or fix the leak
Exit code 1	Application error → read the logs
Can't reach the app	Correct `-p` mapping? App listens on `0.0.0.0`, not `127.0.0.1`? Firewall?
Can't reach another container	Same user-defined network? Using the container name as host? Right port?
Image too big	Slim base, multi-stage, `.dockerignore`, clean caches in the same `RUN`

8.11 More Docker commands and the container ecosystem

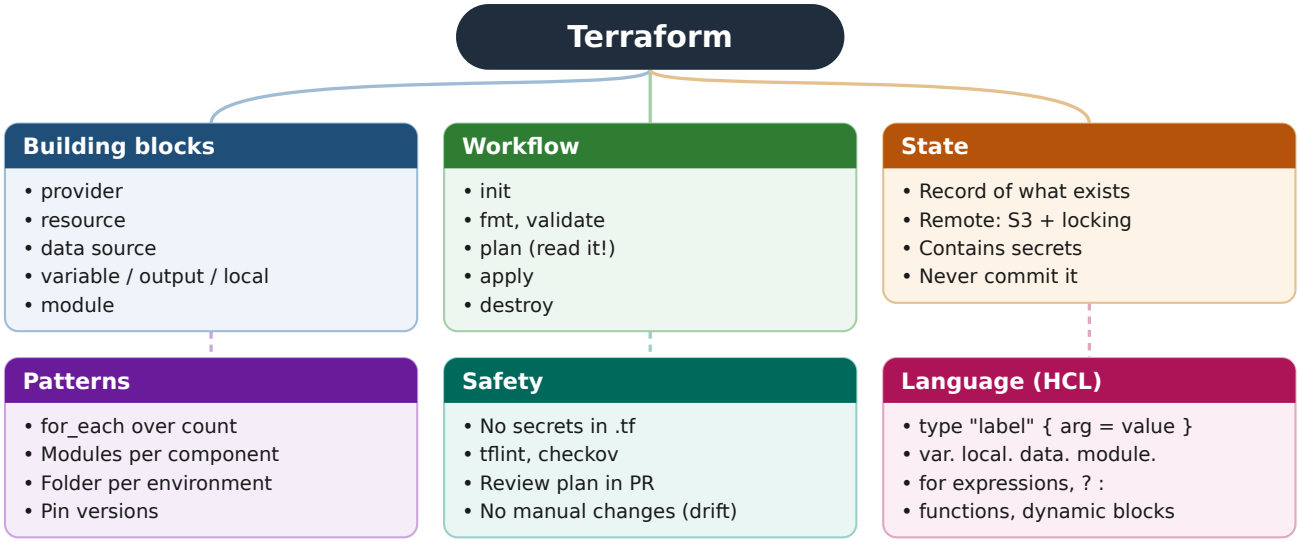
Command	What it does
`docker cp web:/etc/nginx/nginx.conf .`	Copy files out of (or into) a container
`docker save -o app.tar myapp:1.0` / `docker load -i app.tar`	Export/import an image as a file
`docker history myapp:1.0`	Layers and their sizes
`docker build --no-cache -t myapp .`	Rebuild without the cache
`docker buildx build --platform linux/amd64,linux/arm64 -t user/app:1.0 --push .`	Multi-architecture build (e.g. for AWS Graviton ARM)
`docker run --rm --env-file .env --restart unless-stopped ...`	Auto-delete on exit / load env file / restart policy
`docker inspect -f '{{.State.ExitCode}}' web`	Pick one field from inspect

Ecosystem: images follow the **OCI** standard, so they run anywhere. Kubernetes uses **containerd** (not Docker) to run containers. **Podman** is a daemonless Docker alternative with the same commands. **Layer order rule:** put things that change rarely (base image, dependencies) first and code last, so rebuilds reuse the cache.

REMEMBER

- Port mapping is **-p HOST:CONTAINER**.
- Dockerfile order: **base → dependencies → code** (code changes most, so it goes last).
- Exit code **137** = killed (usually out of memory). Exit **1** = application error → read the logs.
- Inside a container, `localhost` means the container itself.

9. Terraform (Infrastructure as Code)



9.1 Concepts

Term	Meaning
Infrastructure as Code (IaC)	Infrastructure defined in code files: versioned, reviewed, repeatable
Declarative	You describe the end state; the tool works out the steps
Provider	A plugin that talks to a platform's API (aws, azurerm, kubernetes)
Resource	Something to create (`aws_instance` , `aws_s3_bucket`)
Data source	Reads existing information (latest AMI, existing VPC)
Variable / output / local	Input / printed result / internal computed value
State	`terraform.tfstate` : Terraform's record of what it created. **Contains sensitive data**
Remote state	State stored in S3 (with locking) so a team shares it safely
Module	A reusable folder of Terraform code
Plan	Preview of changes: `+` create, `~` change, `-` destroy, `-/+` replace

9.2 Workflow commands

Command	What it does
`terraform init`	Download providers/modules, set up the backend
`terraform fmt`	Format the code
`terraform validate`	Check syntax
`terraform plan`	Show what will change. **Always read it**
`terraform apply`	Make the changes (asks for confirmation)

Command	What it does
`terraform destroy`	Delete everything in this configuration
`terraform output`	Show outputs
`terraform state list` / `state show <addr>`	Inspect state
`terraform import <addr> <id>`	Bring an existing resource under Terraform
`terraform plan -out=tfplan` → `apply tfplan`	Apply exactly what was reviewed

9.3 Example

```
# versions.tf
terraform {
  required_version = ">= 1.6"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
  backend "s3" {
    bucket       = "my-tf-state-bucket"
    key          = "dev/network/terraform.tfstate"
    region       = "ap-south-1"
    use_lockfile = true # state locking with S3
    encrypt      = true
  }
}

provider "aws" {
  region = var.region
  default_tags { tags = { Project = "devops-lab", Env = var.env } }
}

# variables.tf
variable "region" {
  type    = string
  default = "ap-south-1"
}

variable "env" {
  type = string
}

# main.tf
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}

resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.main.id # reference creates a dependency
  cidr_block              = "10.0.1.0/24"
  map_public_ip_on_launch = true
}

# outputs.tf
output "vpc_id" { value = aws_vpc.main.id }
```

```
# terraform.tfvars  (values per environment, no secrets)
env = "dev"
```

Module use:

```
module "network" {
  source     = "./modules/network"
  cidr_block = "10.0.0.0/16"
}
```

Other useful features: `count`, `for_each` (create many), `depends_on`, `lifecycle { prevent_destroy = true }`, `sensitive = true` on variables/outputs.

9.4 Project layout

```
infra/
├── modules/
│   └── network/ (main.tf, variables.tf, outputs.tf)
└── envs/
    ├── dev/  (main.tf, backend.tf, terraform.tfvars)
    └── prod/
```

9.5 Security and best practices

- Remote state in an encrypted S3 bucket with locking and versioning. Never commit `*.tfstate` or `.terraform/`.
- No secrets in `.tf` or `.tfvars` files. Read them from Secrets Manager/SSM or environment variables (`TF_VAR_name`).
- Pin provider and module versions. Commit `.terraform.lock.hcl`.
- Review `plan` in pull requests. Run `fmt`, `validate`, `tflint`, and a security scanner (`checkov` / `trivy config`) in CI.
- Small, separate states per environment/component. Least-privilege IAM role for Terraform.
- Don't change Terraform-managed resources by hand (causes **drift**).

9.6 Troubleshooting

Error	Fix
State lock error	Someone else is running it, or a crashed run left a lock. Wait, or `terraform force-unlock <id>` only if sure
Access denied / unauthorized	Check `aws sts get-caller-identity`, profile, region, IAM permissions
Resource already exists	Import it, or change the name
Plan wants to replace something unexpectedly	Read which attribute forces replacement; fix the code or use `lifecycle`
Dependency/order errors	Use references instead of hard-coded IDs; `depends_on` if needed
Drift	`terraform plan` shows it; decide whether code or the real resource is correct

9.7 More Terraform features

```
# Data source: look up the latest Ubuntu AMI instead of hard-coding an ID
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"] # Canonical
  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
  }
}

# for_each: one resource per item in a map (stable names, safe to add/remove)
variable "subnets" {
  default = {
    public-a = "10.0.1.0/24"
    public-b = "10.0.2.0/24"
  }
}

resource "aws_subnet" "this" {
  for_each   = var.subnets
  vpc_id     = aws_vpc.main.id
  cidr_block = each.value
  tags       = { Name = each.key }
}
```

Command / feature	Purpose
`terraform apply -replace=aws_instance.web`	Force one resource to be recreated (replaces old `taint`)
`terraform plan -refresh-only`	Detect drift without changing anything
`terraform console`	Try expressions interactively
`terraform workspace new dev`	Separate state per workspace (simple setups; many teams prefer one folder per environment)
`moved { from = ... to = ... }`	Rename/move resources in code without destroying them
`count` vs `for_each`	`count` for N identical copies; `for_each` for named items (preferred)

OpenTofu is an open-source fork of Terraform with almost the same commands.

9.8 Terraform language (HCL): what is what

Anatomy of a block

```
# block type = resource | label 1 = resource type | label 2 = your name for it
resource "aws_instance" "web" {
  ami           = "ami-0abc123"     # ARGUMENT: name = value
  instance_type = var.instance_type # a value can reference other things
  tags = {                          # map value
    Name = "web-1"
  }
  root_block_device { # NESTED BLOCK (no "=")
    volume_size = 20
  }
}
```

- **Block** = `type "labels" { body }`. **Argument** = `name = value`. **Nested block** = `name { ... }` without `=`.
- The resource's **address** is `aws_instance.web`. Other code reads its attributes, e.g. `aws_instance.web.public_ip`.
- Terraform reads **all `.tf` files in the folder** together; file names are only for humans.

Block types

Block	Purpose	Example
`terraform`	Terraform version, required providers, backend (state location)	`terraform { required_version = ">= 1.6" }`
`provider`	Configures a platform's API (region, credentials from environment/role)	`provider "aws" { region = "ap-south-1" }`
`resource`	Something Terraform **creates and manages**	`resource "aws_s3_bucket" "logs" { ... }`
`data`	**Reads** something that already exists	`data "aws_availability_zones" "available" {}`
`variable`	An **input**	`variable "env" { type = string }`
`locals`	**Computed helper values** inside the module	`locals { name = "${var.env}-web" }`
`output`	A value printed after apply / used by other modules	`output "ip" { value = aws_instance.web.public_ip }`
`module`	Uses a reusable folder of Terraform code	`module "vpc" { source = "./modules/vpc" }`
`moved` / `import`	Refactor without destroying / adopt existing resources	`import` block with `to = aws_s3_bucket.logs` and `id = "my-bucket"`

How to reference things

You write	It means
`var.env`	Input variable `env`
`local.name`	Local value `name`
`aws_vpc.main.id`	Attribute `id` of resource `aws_vpc` named `main`
`data.aws_ami.ubuntu.id`	Attribute of a data source
`module.vpc.vpc_id`	Output `vpc_id` of module `vpc`
`aws_subnet.private[0].id`	Item 0 when the resource uses `count`
`aws_subnet.this["public-a"].id`	Item by key when the resource uses `for_each`
`each.key` / `each.value`	Current item inside a `for_each` resource
`count.index`	Current number (0, 1, 2…) inside a `count` resource
`path.module`	Folder of the current module (for files/templates)

Implicit dependencies: referencing `aws_vpc.main.id` inside a subnet tells Terraform to create the VPC first. Use `depends_on` only when there is no reference.

Data types

Type	Example
`string`	`"t3.micro"`
`number`	`3`
`bool`	`true`
`list(string)`	`["a", "b"]` — ordered, accessed by index `[0]`
`set(string)`	`toset(["a", "b"])` — unique, no order
`map(string)`	`{ env = "dev", team = "ops" }` — accessed by key `["env"]`
`object({...})`	`{ name = string, size = number }` — fixed structure

Variables in detail

```
variable "instance_type" {
  description = "EC2 size for the web tier" # documents the input
  type        = string                      # type constraint
  default     = "t3.micro"                  # optional: no default = required input

  validation { # reject bad values early
    condition     = contains(["t3.micro", "t3.small"], var.instance_type)
    error_message = "Use t3.micro or t3.small."
  }
}

variable "db_password" {
  type      = string
  sensitive = true # hidden in plan/apply output (still stored in state!)
}
```

Ways to set variables (later ones win): `default` → `terraform.tfvars` / `*.auto.tfvars` → `-var-file=prod.tfvars` → `-var="env=prod"`. Environment variable form: `TF_VAR_db_password`.

Expressions

```
locals {
  name        = "${var.project}-${var.env}"                    # string interpolation
  is_prod     = var.env == "prod"                              # comparison → bool
  size        = local.is_prod ? "t3.small" : "t3.micro"        # conditional: cond ? a : b
  upper_names = [for s in var.servers : upper(s)]              # for expression → list
  name_map    = { for s in var.servers : s => "${s}.internal" } # for expression → map
  subnet_ids  = aws_subnet.private[*].id                       # splat: every id
}
```

Useful built-in functions

Function	Example	Result
`length`	`length(["a","b"])`	`2`
`upper` / `lower`	`upper("dev")`	`"DEV"`
`join` / `split`	`join(",", ["a","b"])`	`"a,b"`
`merge`	`merge(local.tags, { Name = "web" })`	Combined map
`concat`	`concat(["a"], ["b"])`	`["a","b"]`

Function	Example	Result
`lookup`	`lookup(var.sizes, "prod", "t3.micro")`	Value or default
`contains`	`contains(["a","b"], "a")`	`true`
`cidrsubnet`	`cidrsubnet("10.0.0.0/16", 8, 1)`	`"10.0.1.0/24"`
`file` / `templatefile`	`templatefile("${path.module}/init.sh.tftpl", { port = 80 })`	File content (with variables)
`try` / `coalesce`	`try(var.cfg.size, "small")`	Fallback when missing
`toset` / `tolist`	`toset(var.names)`	Type conversion

Try any function in `terraform console`.

Meta-arguments (work on every resource)

Meta-argument	Purpose
`count = 3`	Create N copies (`count.index`)
`for_each = toset([...])` or a map	One copy per item (`each.key` , `each.value`) — preferred
`depends_on = [aws_iam_role.x]`	Force an order when there is no reference
`provider = aws.us_east`	Use an aliased provider (e.g. second region)
`lifecycle { ... }`	`create_before_destroy` , `prevent_destroy` , `ignore_changes`

Dynamic block (repeat a nested block from a list):

```
variable "ingress_ports" {
  type    = list(number)
  default = [80, 443]
}

resource "aws_security_group" "web" {
  name   = "web-sg"
  vpc_id = aws_vpc.main.id

  dynamic "ingress" {
    for_each = var.ingress_ports
    content {
      from_port   = ingress.value
      to_port     = ingress.value
      protocol    = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    }
  }
}
```

Complete annotated example: VPC + public subnet + web server

```
# ---------- versions.tf ----------
terraform {
  required_version = ">= 1.6"
  required_providers {
    aws = {
      source  = "hashicorp/aws" # where to download the provider
      version = "~> 5.0"        # any 5.x version
    }
  }
}

provider "aws" {
  region = var.region # credentials come from your AWS CLI profile / IAM role
  default_tags {
    tags = local.common_tags # added to every resource automatically
  }
}

# ---------- variables.tf ----------
variable "region" {
  type    = string
  default = "ap-south-1"
}

variable "env" {
  type        = string
  description = "Environment name, e.g. dev or prod"
}

variable "my_ip_cidr" {
  type        = string
  description = "Your public IP for SSH, e.g. 203.0.113.10/32"
}

# ---------- main.tf ----------
locals {
  name        = "lab-${var.env}"
  common_tags = { Project = "devops-lab", Env = var.env, ManagedBy = "terraform" }
}

data "aws_availability_zones" "available" {
  state = "available" # read the AZ names of this region
}

data "aws_ami" "ubuntu" { # find the newest Ubuntu 24.04 image
  most_recent = true
  owners      = ["099720109477"] # Canonical's account
  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
  }
}

resource "aws_vpc" "main" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_hostnames = true
  tags                 = { Name = "${local.name}-vpc" }
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id # reference → VPC is created first
}

resource "aws_subnet" "public" {
```

```
  vpc_id                  = aws_vpc.main.id
  cidr_block              = cidrsubnet(aws_vpc.main.cidr_block, 8, 1) # 10.0.1.0/24
  availability_zone       = data.aws_availability_zones.available.names[0]
  map_public_ip_on_launch = true
  tags                    = { Name = "${local.name}-public-a" }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"                 # everything else ...
    gateway_id = aws_internet_gateway.igw.id # ... goes to the internet gateway
  }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}

resource "aws_security_group" "web" {
  name   = "${local.name}-web"
  vpc_id = aws_vpc.main.id

  ingress {
    description = "HTTP from anywhere"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    description = "SSH only from my IP"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.my_ip_cidr]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1" # all protocols
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_instance" "web" {
  ami                    = data.aws_ami.ubuntu.id
  instance_type          = "t3.micro"
  subnet_id              = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.web.id]
  user_data              = <<-EOT
    #!/bin/bash
    apt-get update -y
    apt-get install -y nginx
    systemctl enable --now nginx
  EOT
  tags                   = { Name = "${local.name}-web" }
}

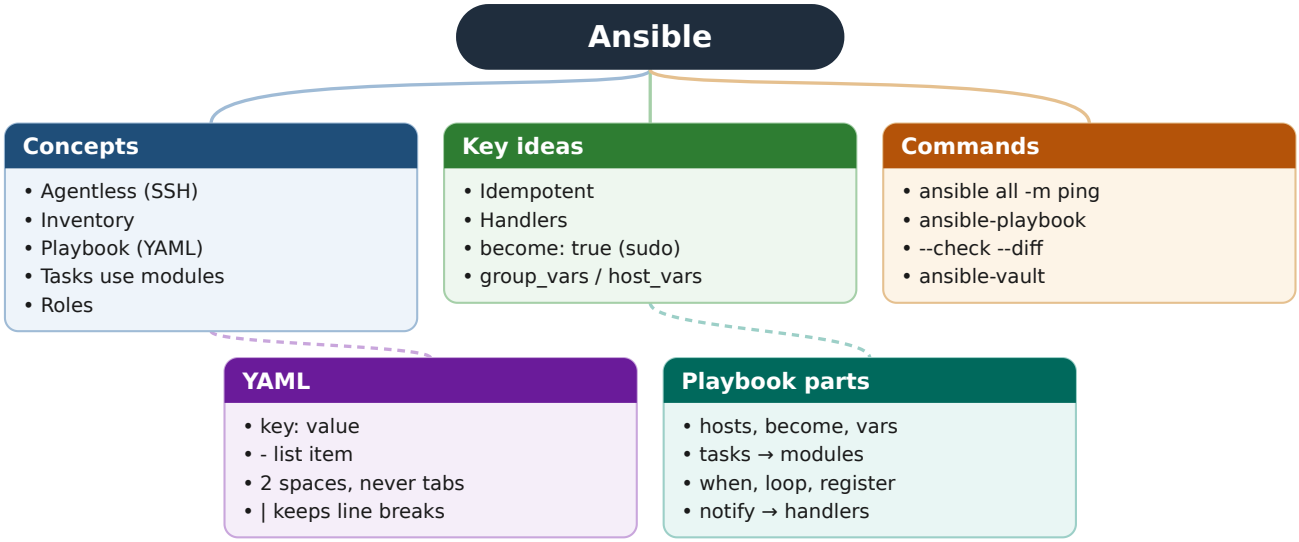
# ---------- outputs.tf ----------
output "web_url" {
  description = "Open this in a browser"
  value       = "http://${aws_instance.web.public_ip}"
}
```

Run it: `terraform init` → `terraform plan -var="env=dev" -var="my_ip_cidr=<your-ip>/32"` → `terraform apply ...` → open the URL → **`terraform destroy ...`** when finished.

Naming conventions: snake_case names (`aws_instance.web_server`), one purpose per file (`versions.tf`, `variables.tf`, `main.tf`, `outputs.tf`), `this` as the name when a module has only one resource of a type, run `terraform fmt` before every commit.

> **REMEMBER**
>
> - The loop: **write → init → plan → apply**.
> - Plan symbols: **+** create · **~** update · **-** destroy · **-/+** replace (check why!).

10. Ansible (Configuration Management)



Configuration management = keeping servers configured the same way (packages, files, users, services) automatically. **Terraform builds the servers; Ansible configures what's inside them.**

Term	Meaning
Agentless	Connects over SSH; nothing to install on target servers
Inventory	List of servers and groups
Playbook	YAML file with tasks to run on hosts
Task / module	One action, e.g. `apt`, `copy`, `template`, `service`, `user`
Role	A reusable, structured bundle of tasks, templates and variables
Idempotent	Running it again changes nothing if the server is already correct
Handler	A task that runs only when notified (e.g. restart nginx after config changes)
Ansible Vault	Encrypts secrets in Ansible files

```
# inventory.ini
[web]
10.0.1.10
10.0.1.11

[web:vars]
ansible_user=ubuntu
```

```
# site.yml
- name: Configure web servers
  hosts: web
  become: true                    # use sudo
  tasks:
    - name: Install nginx
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true
    - name: Deploy config
      ansible.builtin.template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
      notify: Restart nginx
    - name: Ensure nginx is running
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
  handlers:
    - name: Restart nginx
      ansible.builtin.service:
        name: nginx
        state: restarted
```

Command	What it does
`ansible all -i inventory.ini -m ping`	Test the connection to all hosts
`ansible-playbook -i inventory.ini site.yml`	Run a playbook
`ansible-playbook ... --check --diff`	Dry run, show what would change
`ansible-vault encrypt secrets.yml`	Encrypt a secrets file

More Ansible	Purpose
`ansible-galaxy collection install amazon.aws`	Install community/vendor modules
`ansible web -m setup`	Show facts (OS, IPs) gathered from hosts
`group_vars/`, `host_vars/`	Variables per group / per host
`--limit web1`, `--tags nginx`	Run on one host / only tagged tasks
Dynamic inventory	Build the host list from AWS automatically (by tags)

10.1 YAML: what is what

YAML is used by Ansible, Kubernetes, Docker Compose, GitHub Actions and Helm, so learn it once.

```
# Comment
name: web-server          # key: value   (a space after the colon is required)
port: 8080                # number
enabled: true             # boolean (true/false)
version: "1.10"           # quote values that look like numbers but are text
empty: null               # no value

servers:                  # LIST: each item starts with "- "
  - web1
  - web2

database:                 # MAP (dictionary): nested keys, indented
  host: db.internal
  port: 5432

users:                    # list of maps
  - name: alice
    groups: [admin, dev]  # inline list
  - name: bob
    groups: [dev]

script: |                 # | keeps line breaks (multi-line text, scripts)
  echo "line 1"
  echo "line 2"
description: >            # > folds lines into one long line
  This is a long
  sentence.
```

Rules: indentation = structure, **spaces only (never tabs)**, usually 2 spaces per level, items in the same list must line up. Check a file with `yamllint file.yml` or `python3 -c "import yaml,sys; yaml.safe_load(open(sys.argv[1]))" file.yml`.

10.2 Playbook anatomy: what is what

```
---                                  # optional start-of-document marker
- name: Configure web servers        # PLAY: a name for humans
  hosts: web                         # which inventory group to run on
  become: true                       # use sudo for tasks
  vars:                              # VARIABLES for this play
    app_port: 8080
    packages:
      - nginx
      - curl

  tasks:                             # TASKS run in order, top to bottom
    - name: Install packages         # always name tasks: shown in the output
      ansible.builtin.apt:           # MODULE (fully qualified name)
        name: "{{ packages }}"       # {{ }} = Jinja2 variable (quote it)
        state: present               # desired state, not a command → idempotent
        update_cache: true

    - name: Deploy nginx site config
      ansible.builtin.template:      # renders a Jinja2 template file with variables
        src: templates/site.conf.j2
        dest: /etc/nginx/sites-available/default
        mode: "0644"
      notify: Reload nginx           # trigger the HANDLER only if this task changed something

    - name: Check if the app answers
      ansible.builtin.uri:
        url: "http://localhost:{{ app_port }}/health"
        status_code: 200
      register: health               # save the result in a variable
      ignore_errors: true

    - name: Show the result
      ansible.builtin.debug:
        msg: "Health status: {{ health.status | default('unknown') }}"   # | = Jinja2 filter

    - name: Create users
      ansible.builtin.user:
        name: "{{ item }}"
        groups: www-data
        append: true
      loop:                          # LOOP: run the task once per item
        - alice
        - bob

    - name: Only on Ubuntu
      ansible.builtin.debug:
        msg: "This is {{ ansible_facts['distribution'] }}"   # FACTS gathered from the host
      when: ansible_facts['distribution'] == "Ubuntu"         # CONDITION

  handlers:                          # HANDLERS run once at the end, only when notified
    - name: Reload nginx
      ansible.builtin.service:
        name: nginx
        state: reloaded
```

Template file `templates/site.conf.j2` (Jinja2 = text with `{{ variables }}` and `{% logic %}`):

```
server {
    listen 80;
    location / {
        proxy_pass http://127.0.0.1:{{ app_port }};
    }
}
```

Element	Meaning
Play	Maps a group of hosts to a list of tasks
Task	One step, using one module
Module	The tool that does the work (`apt` , `copy` , `template` , `service` , `user` , `file` , `uri` , `command`)
`state`	Desired result: `present` / `absent` , `started` / `stopped` / `restarted`
`{{ var }}`	Insert a variable (Jinja2)
`register`	Save a task's result
`when`	Run the task only if the condition is true
`loop` / `item`	Repeat a task for each item
`notify` / `handlers`	Run a follow-up (e.g. restart) only when something changed
`become`	Run with sudo
Facts (`ansible_facts`)	Information collected from each host (OS, IPs, memory)

Role layout (reusable unit, created with `ansible-galaxy init nginx`):

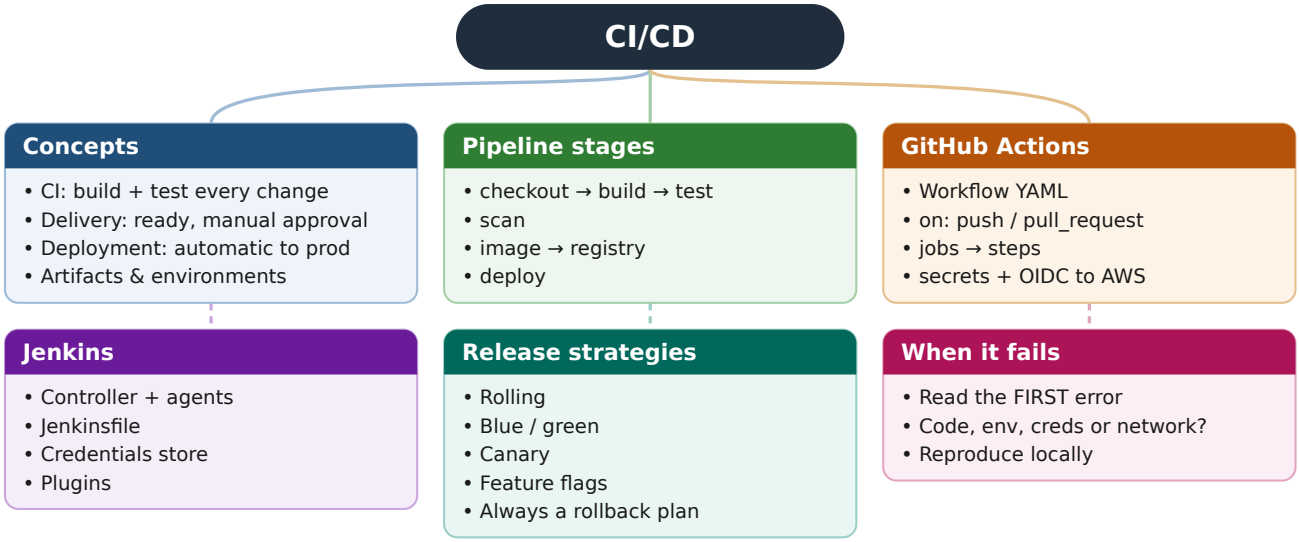
```
roles/nginx/
├── tasks/main.yml        # the tasks
├── handlers/main.yml     # handlers
├── templates/            # .j2 files
├── defaults/main.yml     # default variables (easy to override)
└── vars/main.yml         # fixed variables
```

Use it in a playbook with `roles: [nginx]` . **Avoid `command` / `shell` modules** when a real module exists: real modules are idempotent and report "changed" correctly.

REMEMBER

- **Terraform builds the house; Ansible furnishes it.**
- Idempotent = running it twice changes nothing the second time.

11. CI/CD



11.1 Concepts

Term	Meaning
Continuous Integration (CI)	Every change is automatically built and tested when pushed / on every PR
Continuous Delivery	Every passing change is ready to release; production deploy needs a manual approval
Continuous Deployment	Every passing change goes to production automatically
Pipeline	The automated sequence of stages
Stage / job / step	A phase (build, test) / a unit of work on one runner / a single command
Runner / agent	The machine that executes the jobs
Artifact	An output kept from the pipeline (binary, test report, Docker image)
Trigger	What starts the pipeline: push, PR, schedule, manual
Environment	A deployment target (dev, staging, prod), often with approval rules

```
Code push → Build → Test → Scan → Docker image → Push to registry
          → Deploy dev → (approval) → Deploy prod
```

11.2 GitHub Actions

Term	Meaning
Workflow	A YAML file in `.github/workflows/`
Event (`on:`)	Trigger: `push` , `pull_request` , `workflow_dispatch` (manual), `schedule`
Job	Runs on a runner (`runs-on: ubuntu-latest`). Jobs run in parallel unless `needs:`

Term	Meaning
Step	`uses:` a ready-made action, or `run:` a shell command
Secrets / variables	Repo or environment settings, used as `${{ secrets.NAME }}`
OIDC	The workflow gets short-lived AWS credentials by assuming an IAM role. **No stored AWS keys**

```
# .github/workflows/ci.yml
name: ci
on:
  push:
    branches: [main]
  pull_request:

permissions:
  contents: read
  id-token: write          # needed for OIDC to AWS

jobs:
  build-test-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install and test
        run: |
          pip install -r requirements.txt
          pytest

      - name: Configure AWS credentials (OIDC)
        if: github.ref == 'refs/heads/main'
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/github-actions-ecr
          aws-region: ap-south-1

      - name: Log in to ECR
        if: github.ref == 'refs/heads/main'
        id: ecr
        uses: aws-actions/amazon-ecr-login@v2

      - name: Build, scan, push image
        if: github.ref == 'refs/heads/main'
        env:
          IMAGE: ${{ steps.ecr.outputs.registry }}/myapp:${{ github.sha }}
        run: |
          docker build -t "$IMAGE" .
          trivy image --exit-code 1 --severity CRITICAL "$IMAGE"
          docker push "$IMAGE"
```

Good practice: least `permissions:`, pin action versions, use OIDC instead of keys, protected `main` with required checks, environments with approvals for prod.

11.3 Jenkins

Term	Meaning
Controller	The Jenkins server: UI, scheduling, configuration
Agent	A machine that runs builds (keep builds off the controller)
Job / Pipeline	A configured task / a pipeline defined in code
Jenkinsfile	Pipeline as code, stored in the repo
Plugins	Add features (Git, Docker, credentials, Kubernetes)
Credentials store	Encrypted secrets used in pipelines via `withCredentials` / `credentials()`

```
// Jenkinsfile (declarative)
pipeline {
  agent any
  environment {
    IMAGE = "myuser/myapp:${env.BUILD_NUMBER}"
  }
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Test')     { steps { sh 'pip install -r requirements.txt && pytest' } }
    stage('Build')    { steps { sh 'docker build -t $IMAGE .' } }
    stage('Push') {
      steps {
        withCredentials([usernamePassword(credentialsId: 'dockerhub',
                         usernameVariable: 'USER', passwordVariable: 'PASS')]) {
          sh 'echo "$PASS" | docker login -u "$USER" --password-stdin && docker push $IMAGE'
        }
      }
    }
    stage('Deploy') {
      when { branch 'main' }
      steps { sh './deploy.sh $IMAGE' }
    }
  }
  post {
    failure { echo 'Pipeline failed: check the stage logs' }
    always  { cleanWs() }
  }
}
```

Run Jenkins locally: `docker run -d -p 8080:8080 -v jenkins_home:/var/jenkins_home jenkins/jenkins:lts`. The first admin password is in `/var/jenkins_home/secrets/initialAdminPassword`.

	GitHub Actions	Jenkins
Hosting	Managed by GitHub (or self-hosted runners)	You install and maintain it
Config	YAML in `.github/workflows`	Groovy `Jenkinsfile`
Best for	Repos on GitHub, quick setup	Complex/on-prem setups, many legacy enterprises

11.4 Pipeline failure checklist

1. Which stage failed? Read the **first** error in the log, not the last.
2. Is it the code (tests failing), the environment (missing tool/version), credentials/permissions, or the network?
3. Does it pass locally with the same command and versions?

4. Did anything change recently (dependency version, secret expired, runner image)?
5. Fix the cause and push again. Never disable tests to get green.

11.5 Deployment strategies

Strategy	How it works	Pros	Cons
Recreate	Stop old version, start new	Simple	Downtime
Rolling update	Replace instances a few at a time	No downtime, default in Kubernetes	Old and new run together for a while
Blue/green	Run new version (green) next to old (blue), switch traffic at once	Instant switch and rollback	Double resources during release
Canary	Send a small % of traffic to the new version, watch metrics, then increase	Limits the blast radius	Needs good monitoring and traffic splitting
Feature flags	New code is deployed but switched off; turn on per user/percentage	Separates deploy from release	Flag management overhead

Rollback plan: every deployment needs one (previous image tag, `kubectl rollout undo`, `helm rollback`, `git revert` in GitOps). Database changes must be backward-compatible so you can roll back the app.

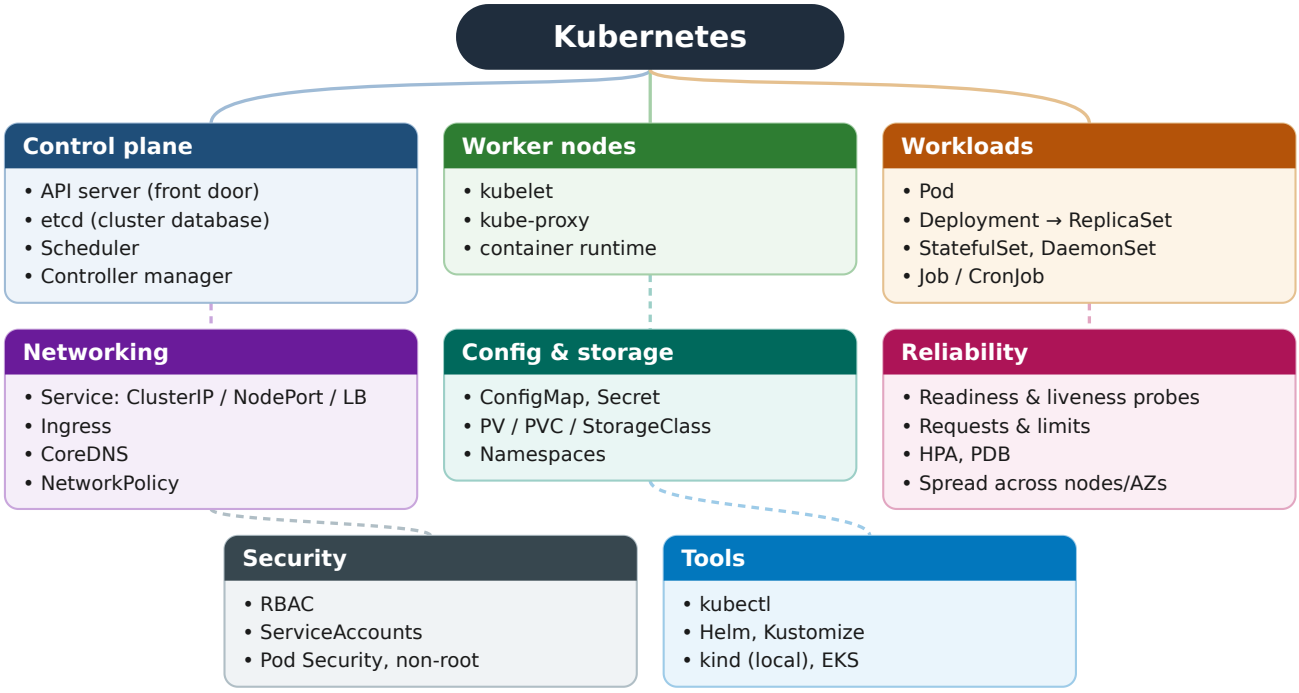
11.6 More GitHub Actions features

Feature	Example / purpose
Caching	`actions/cache` or `cache: pip` in setup actions → faster builds
Matrix builds	`strategy: matrix: python: ["3.11", "3.12"]` → test several versions in parallel
Job dependencies	`needs: test` → deploy only after tests pass
Environments	`environment: production` → required reviewers and environment-specific secrets
Reusable workflows	`uses: org/repo/.github/workflows/build.yml@v1` → one standard pipeline for many repos
Artifacts	`actions/upload-artifact` / `download-artifact` → pass build outputs between jobs
Concurrency	`concurrency: deploy-prod` → only one deploy at a time
Path filters	`on: push: paths: ["app/**"]` → run only when relevant files change

REMEMBER

- **Code → Build → Test → Scan → Image → Registry → Deploy.**
- Prefer **OIDC** (short-lived credentials) over stored cloud keys.
- Never disable tests to make a pipeline green.

12. Kubernetes



12.1 Architecture

```
                CONTROL PLANE
┌──────────────┬──────────┬─────────────┬──────────────────────┐
│ API server   │   etcd   │ scheduler   │ controller-manager   │
└──────┬───────┴──────────┴─────────────┴──────────────────────┘
       │  (kubectl talks to the API server)
┌──────┴──────────────┐    ┌──────────────────────┐
│ WORKER NODE 1       │    │ WORKER NODE 2        │
│ kubelet, kube-proxy │    │ kubelet, kube-proxy  │
│ container runtime   │    │ container runtime    │
│ [pod] [pod]         │    │ [pod] [pod]          │
└─────────────────────┘    └──────────────────────┘
```

Component	Job
API server	Front door of the cluster. Everything goes through it
etcd	Key-value database storing the whole cluster state
Scheduler	Chooses a node for each new pod
Controller manager	Loops that make the actual state match the desired state
kubelet	Agent on each node that starts and watches pods
kube-proxy	Sets up Service networking on each node
Container runtime	Runs containers (containerd)

Core idea: desired state. You declare what you want in YAML; Kubernetes keeps making it true (restarts, reschedules, scales).

12.2 Objects

Object	Purpose
Pod	Smallest unit: one or more containers sharing network and storage. Temporary
ReplicaSet	Keeps N identical pods running
Deployment	Manages ReplicaSets: rolling updates and rollbacks. Use this for stateless apps
StatefulSet	Pods with stable names and storage (databases)
DaemonSet	One pod on every node (log/monitoring agents)
Job / CronJob	Run to completion / on a schedule
Service	Stable IP and DNS name in front of a set of pods
Ingress	HTTP(S) routing from outside to Services by host/path (needs an Ingress controller)
ConfigMap	Non-secret configuration
Secret	Sensitive data (base64-encoded, **not encrypted by default**: enable encryption, restrict access)
Namespace	Logical separation inside a cluster (dev, team-a)
PersistentVolume (PV) / Claim (PVC)	A piece of storage / a pod's request for storage
StorageClass	How storage is created dynamically (e.g. AWS EBS gp3)
ServiceAccount	Identity for pods
HPA	Horizontal Pod Autoscaler: changes replica count based on metrics

12.3 YAML basics

```
key: value            # mapping
list:                 # list
  - item1
  - item2
nested:
  child: value        # indentation = structure (spaces only, never tabs)
```

Every Kubernetes object has: `apiVersion`, `kind`, `metadata` (name, labels), `spec` (desired state).

12.4 Deployment + Service example

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  labels: { app: web }
spec:
  replicas: 3                       # how many pods
  selector:
    matchLabels: { app: web }       # which pods this Deployment owns
  strategy:
    type: RollingUpdate
    rollingUpdate: { maxSurge: 1, maxUnavailable: 0 }
  template:                         # the pod blueprint
    metadata:
      labels: { app: web }          # must match the selector
    spec:
      containers:
        - name: web
          image: myuser/myapp:1.4.2 # pinned version, not latest
          ports:
            - containerPort: 8000
          envFrom:
            - configMapRef: { name: web-config }
            - secretRef:    { name: web-secrets }
          resources:
            requests: { cpu: 100m, memory: 128Mi }   # used for scheduling
            limits:   { cpu: 500m, memory: 256Mi }   # hard cap (over memory limit = OOMKilled)
          readinessProbe:           # ready to receive traffic?
            httpGet: { path: /health, port: 8000 }
            initialDelaySeconds: 5
          livenessProbe:            # still alive? (restart if not)
            httpGet: { path: /health, port: 8000 }
            initialDelaySeconds: 15
          securityContext:
            runAsNonRoot: true
            allowPrivilegeEscalation: false
---
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  type: ClusterIP                   # internal only
  selector: { app: web }            # send traffic to pods with this label
  ports:
    - port: 80                      # Service port
      targetPort: 8000              # container port
```

Service type	Reachable from
ClusterIP (default)	Inside the cluster only. DNS: `web.<namespace>.svc.cluster.local`
NodePort	Each node's IP on a port 30000–32767
LoadBalancer	A cloud load balancer (AWS NLB/ALB)

Probes: readiness = should it get traffic; liveness = should it be restarted; startup = give slow apps time to start. **CPU units:** `1000m` = 1 core. **Memory:** `Mi`, `Gi`.

12.5 ConfigMap, Secret, Ingress, PVC

```
apiVersion: v1
kind: ConfigMap
metadata: { name: web-config }
data:
  APP_ENV: "prod"
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: { name: web }
spec:
  ingressClassName: nginx
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service: { name: web, port: { number: 80 } }
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: data }
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: gp3
  resources:
    requests: { storage: 10Gi }
```

Create secrets from the command line (not committed to Git): `kubectl create secret generic web-secrets --from-literal=DB_PASSWORD=...` or use External Secrets / Sealed Secrets.

12.6 kubectl commands

Command	What it does
`kubectl get nodes`	List nodes
`kubectl get pods -A` / `-n dev`	Pods in all namespaces / one namespace
`kubectl get pods -o wide`	Extra info (node, IP)
`kubectl get all`	Common objects in the namespace
`kubectl apply -f file.yaml`	Create or update from YAML (declarative)
`kubectl delete -f file.yaml`	Delete what the file defines
`kubectl describe pod <name>`	Details and **Events** (first place to look)
`kubectl logs <pod>` / `-f` / `--previous`	Logs / follow / from the crashed container
`kubectl exec -it <pod> -- sh`	Shell inside a pod
`kubectl port-forward svc/web 8080:80`	Access a service from your laptop
`kubectl rollout status deploy/web`	Watch a rollout
`kubectl rollout history deploy/web`	Revision history
`kubectl rollout undo deploy/web`	Roll back

Command	What it does
`kubectl scale deploy/web --replicas=5`	Scale manually
`kubectl set image deploy/web web=myapp:1.5.0`	Update the image
`kubectl get events --sort-by=.lastTimestamp`	Recent cluster events
`kubectl top pods` / `top nodes`	CPU/memory use (needs metrics-server)
`kubectl config get-contexts` / `use-context`	Which cluster you're talking to
`kubectl explain deployment.spec`	Built-in documentation for fields

Local clusters for practice: **kind** (`kind create cluster`), minikube, k3d.

12.7 Scaling, RBAC, security

HPA: `kubectl autoscale deploy web --cpu-percent=60 --min=2 --max=10` (requires metrics-server and resource requests).

RBAC:

Object	Meaning
Role / ClusterRole	Allowed verbs (get, list, create...) on resources, in one namespace / cluster-wide
RoleBinding / ClusterRoleBinding	Gives a Role to a user, group or ServiceAccount

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { name: pod-reader, namespace: dev }
rules:
  - apiGroups: [""]
    resources: ["pods", "pods/log"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: read-pods, namespace: dev }
subjects:
  - kind: User
    name: alice
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Check permissions: `kubectl auth can-i delete pods -n dev --as alice`.

Security checklist: RBAC least privilege, non-root containers, Pod Security Standards (`restricted`), NetworkPolicies (default deny, then allow what's needed), resource limits, scanned and pinned images, secrets encrypted and access-controlled, no `privileged: true`.

12.8 Pod troubleshooting

Status	Usual cause	First command
Pending	Not enough CPU/memory, node selector/taints, PVC not bound	`kubectl describe pod` → Events
ImagePullBackOff / ErrImagePull	Wrong image name/tag, private registry without credentials	`kubectl describe pod`
CrashLoopBackOff	App starts and crashes: bad config, missing env/secret, wrong command	`kubectl logs <pod> --previous`
OOMKilled	Memory limit too low or a memory leak	`kubectl describe pod` (Last State)
Running but not Ready	Readiness probe failing	`kubectl describe pod`, check the probe path/port
Service not reachable	Selector labels don't match pods → no endpoints; wrong targetPort	`kubectl get endpoints <svc>`
Ingress 404/502	Wrong host/path, wrong service name/port, no Ingress controller	`kubectl describe ingress`

Method: `get` → `describe` (Events) → `logs` (and `--previous`) → `exec` inside to test → check Service endpoints → check node resources.

12.9 Helm

Term	Meaning
Helm	Package manager for Kubernetes
Chart	A package of templated Kubernetes YAML
`values.yaml`	Default settings; override per environment
Release	One installed instance of a chart
Repository	Where charts are published

```
mychart/
├── Chart.yaml          # name, version
├── values.yaml         # replicaCount: 2, image.tag: "1.4.2"
└── templates/
    ├── deployment.yaml # uses {{ .Values.replicaCount }}, {{ .Values.image.tag }}
    └── service.yaml
```

Command	What it does
`helm create mychart`	Create a chart skeleton
`helm repo add bitnami https://charts.bitnami.com/bitnami` / `helm repo update`	Add / refresh a repo
`helm install web ./mychart -f values-prod.yaml -n prod`	Install a release
`helm upgrade --install web ./mychart --set image.tag=1.5.0`	Upgrade (or install)
`helm list -A`	List releases

Command	What it does
`helm history web` / `helm rollback web 2`	History / roll back to revision 2
`helm template ./mychart`	Render the YAML locally without installing
`helm lint ./mychart`	Check the chart
`helm uninstall web`	Remove the release

12.10 Amazon EKS

Term	Meaning
EKS	Managed Kubernetes control plane by AWS (billed per hour)
Managed node group	EC2 worker nodes managed by EKS
Fargate profile	Run pods without managing nodes
VPC CNI	Pods get real VPC IP addresses
AWS Load Balancer Controller	Creates ALBs/NLBs for Ingress and Services
IRSA / EKS Pod Identity	Give pods an IAM role, without keys
Access entries	Map IAM users/roles to Kubernetes permissions

Command	What it does
`aws eks update-kubeconfig --name my-cluster --region ap-south-1`	Connect kubectl to the cluster
`eksctl create cluster --name lab --nodes 2`	Quick cluster for labs
`eksctl delete cluster --name lab`	**Delete after the lab**

EKS costs money every hour (control plane + nodes + load balancers + NAT). Build with Terraform, use it, and destroy it the same day in labs.

12.11 Scheduling and availability

Feature	Purpose
nodeSelector / node affinity	Run pods only on certain nodes (e.g. GPU nodes)
Pod anti-affinity / topologySpreadConstraints	Spread replicas across nodes/AZs so one failure doesn't take all pods
Taints and tolerations	A tainted node repels pods unless they tolerate the taint (dedicated nodes)
PodDisruptionBudget (PDB)	Keeps a minimum number of pods running during node drains/upgrades
Init containers	Run setup steps before the main container starts
Sidecar containers	Helper container in the same pod (log shipper, proxy)
`kubectl drain <node>` / `cordon` / `uncordon`	Safely empty a node for maintenance / stop new pods / allow again

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: { name: web-pdb }
spec:
  minAvailable: 2
  selector:
    matchLabels: { app: web }
```

12.12 NetworkPolicy (firewall between pods)

```
# 1) Deny all incoming traffic to pods in the namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: default-deny-ingress, namespace: prod }
spec:
  podSelector: {}
  policyTypes: [Ingress]
---
# 2) Allow only the web pods to reach the database on 5432
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: allow-web-to-db, namespace: prod }
spec:
  podSelector:
    matchLabels: { app: db }
  ingress:
    - from:
        - podSelector:
            matchLabels: { app: web }
      ports:
        - protocol: TCP
          port: 5432
```

NetworkPolicies only work if the cluster's CNI supports them (Calico, Cilium; on EKS enable network policy in the VPC CNI).

12.13 Kustomize (environment overlays without templates)

```
k8s/
├── base/            (deployment.yaml, service.yaml, kustomization.yaml)
└── overlays/
    ├── dev/         kustomization.yaml → replicas: 1, image tag dev
    └── prod/        kustomization.yaml → replicas: 3, image tag 1.4.2
```

Apply with `kubectl apply -k k8s/overlays/prod`. **Helm vs Kustomize:** Helm = templating + packaging + releases; Kustomize = patching plain YAML. Both work with Argo CD.

12.14 More kubectl

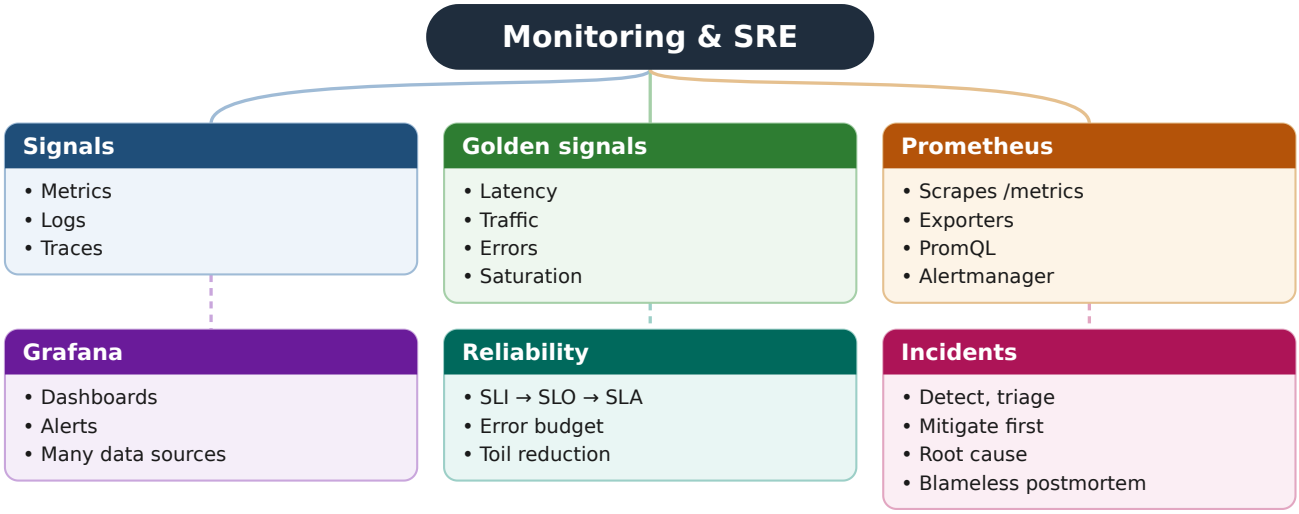
Command	Purpose
`kubectl get deploy web -o yaml`	Full live object as YAML
`kubectl diff -f file.yaml`	What would change before applying
`kubectl create deployment web --image=nginx --dry-run=client -o yaml > web.yaml`	Generate starter YAML

Command	Purpose
`kubectl config set-context --current --namespace=dev`	Change default namespace
`kubectl run tmp --rm -it --image=busybox -- sh`	Temporary debug pod (test DNS: `nslookup web`)
`kubectl debug -it <pod> --image=busybox`	Attach a debug container to a running pod
`kubectl get pods -l app=web`	Filter by label
`kubectl api-resources`	All resource types in the cluster

REMEMBER

- Debug order: **get → describe (Events) → logs --previous → exec → get endpoints**.
- **Labels connect everything**: Deployment selector ↔ pod labels ↔ Service selector.
- **Readiness** = send traffic? **Liveness** = restart? **Requests** = scheduling, **limits** = hard cap.

13. Monitoring & Observability



13.1 Concepts

Term	Meaning
Monitoring	Watching known signals and alerting when something is wrong
Observability	Being able to understand **why** a system behaves a certain way from its outputs
Metrics	Numbers over time (CPU %, requests/sec, error rate)
Logs	Timestamped text records of events
Traces	The path of one request through many services, with timing per step
Alert	A notification when a rule is broken (sent to email, Slack, PagerDuty)
Dashboard	Visual panels of metrics
SLI	Service Level Indicator: what you measure (e.g. % of successful requests)
SLO	Service Level Objective: your target (e.g. 99.9% success over 30 days)
SLA	A promise to customers, with penalties
Error budget	Allowed failure = 100% − SLO (99.9% → about 43 minutes per month)

Golden signals: latency, traffic, errors, saturation. **USE method (resources):** Utilization, Saturation, Errors. **RED method (services):** Rate, Errors, Duration.

13.2 Prometheus

- **Pull model:** Prometheus scrapes `/metrics` endpoints on a schedule and stores time series.
- **Exporters** expose metrics for things that don't have them: node_exporter (Linux), blackbox_exporter (HTTP checks), database exporters.
- **Alertmanager** groups alerts and sends notifications.
- On Kubernetes, install everything with the **kube-prometheus-stack** Helm chart (Prometheus, Alertmanager, Grafana, exporters).

Metric type	Meaning
Counter	Only goes up (total requests) — use `rate()`
Gauge	Goes up and down (memory in use)
Histogram	Buckets of values (request durations) — use for percentiles

PromQL examples:

```
up == 0                                                    # targets that are down
rate(http_requests_total[5m])                              # requests per second
sum(rate(http_requests_total{status=~"5.."}[5m]))
  / sum(rate(http_requests_total[5m]))                     # error ratio
# p95 latency (95% of requests are faster than this)
histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)                        # CPU %
```

Alert rule:

```
groups:
  - name: app
    rules:
      - alert: HighErrorRate
        expr: |
          sum(rate(http_requests_total{status=~"5.."}[5m]))
            / sum(rate(http_requests_total[5m])) > 0.05
        for: 5m
        labels: { severity: critical }
        annotations:
          summary: "More than 5% of requests are failing"
```

13.3 Grafana

- Connects to data sources (Prometheus, Loki, CloudWatch) and builds dashboards and alerts.
- Import community dashboards (e.g. Node Exporter Full) and build your own app dashboard: request rate, error rate, latency, CPU/memory.
- Port-forward on K8s: `kubectl port-forward svc/<grafana-svc> 3000:80 -n monitoring`. Admin password comes from a Kubernetes Secret.

13.4 Logging and tracing

Tool	Use
Loki + Promtail / Grafana Alloy	Collect and search logs, viewed in Grafana
ELK / OpenSearch	Elasticsearch/OpenSearch + Logstash + Kibana: log search
CloudWatch Logs + Logs Insights	AWS central logs and queries
OpenTelemetry	Standard way to produce metrics, logs and traces from apps
Jaeger / Tempo / AWS X-Ray	Store and view traces

Good logs: structured (JSON), with timestamp, level, request ID, no secrets or personal data.

13.5 Troubleshooting with monitoring

1. Alert fires → open the dashboard. **What changed and when?** (deploy, traffic spike, dependency down)
2. Golden signals: errors up? latency up? traffic up? saturation (CPU, memory, disk, connections)?
3. Narrow it down: which service, which endpoint, which node/pod?
4. Logs for that time and component → find the error.
5. Traces → which step is slow.
6. Mitigate first (roll back, scale up), then find the root cause, then write a short post-incident review.

13.6 SRE practices

Practice	Meaning
SLOs and error budgets	Reliability target agreed with the business; if the budget is used up, slow down releases and fix reliability
Toil reduction	Automate repetitive manual work (SREs aim to keep toil below ~50% of their time)
Alert on symptoms	Alert on what users feel (errors, latency), not on every CPU spike. Every alert must be actionable
Runbooks	Step-by-step instructions linked from each alert
Capacity planning	Forecast growth; load test before big events
Blameless postmortems	After an incident, focus on what failed in the system and process, not on who
Chaos engineering	Break things on purpose in a controlled way to find weaknesses

13.7 Incident management

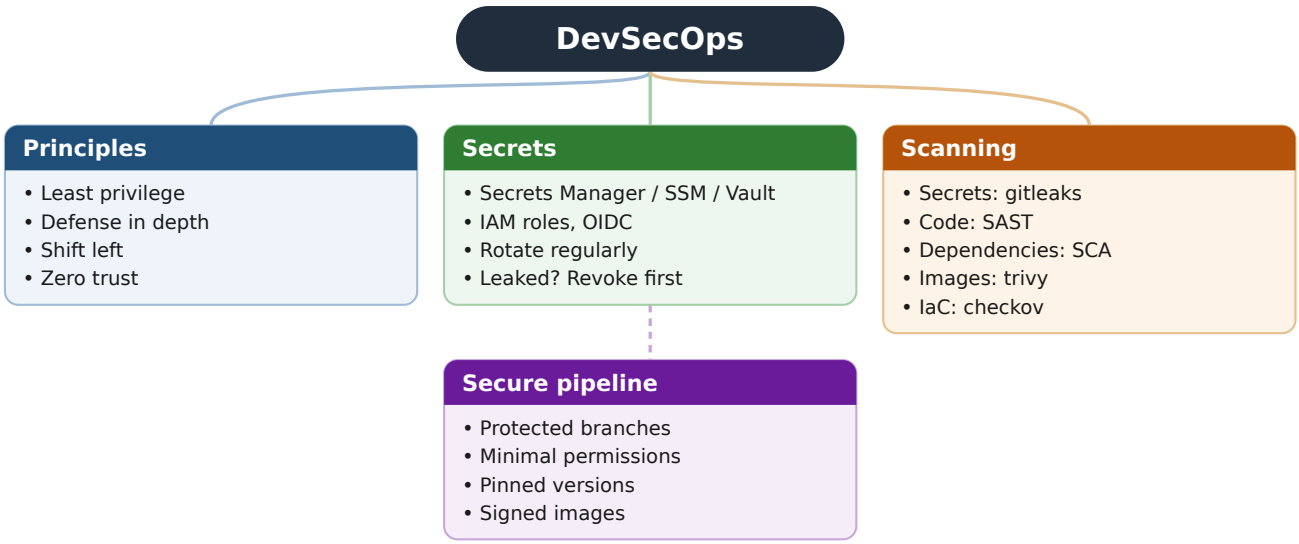
Step	What happens
Detect	Alert fires or a user reports a problem
Triage	Decide severity (SEV1 = major outage … SEV4 = minor) and assign an incident commander
Communicate	Status updates to stakeholders on a fixed schedule
Mitigate	Restore service first: roll back, fail over, scale up, disable a feature
Resolve	Find and fix the root cause
Learn	Postmortem: timeline, impact, root cause, action items with owners

Key metrics: MTTD (time to detect), MTTR (time to recover), change failure rate, deployment frequency, lead time for changes (the last four are the **DORA metrics**).

REMEMBER

- **SLI** = what you measure · **SLO** = your target · **SLA** = promise to customers.
- Alert on what users feel; every alert must be actionable.
- During an incident: **restore service first, investigate second**.

14. DevSecOps (Security Throughout)



14.1 Principles

Principle	Meaning
Least privilege	Every user, service and pipeline gets only the permissions it needs
Defense in depth	Several layers of protection, so one failure isn't fatal
Shift left	Find security problems early (in code and PRs), not in production
Zero trust	Never trust by network location; always authenticate and authorize
Immutable infrastructure	Replace servers/containers instead of patching them by hand

14.2 Secrets management

- Never hardcode secrets or commit them (`.env`, keys, tokens, `.tfstate`).
- Store them in AWS Secrets Manager, SSM Parameter Store, HashiCorp Vault, or CI secret stores.
- Prefer short-lived credentials: IAM roles, OIDC for CI, IRSA/Pod Identity for Kubernetes.
- Rotate secrets. If one leaks, **revoke it first**, then clean up.

14.3 Security scanning in the pipeline

Scan	What it finds	Tools
Secret scanning	Passwords/keys in code and history	gitleaks, GitHub secret scanning, trufflehog
SAST	Insecure code patterns	Semgrep, CodeQL, SonarQube
Dependency scanning (SCA)	Libraries with known vulnerabilities (CVEs)	Dependabot, `pip-audit`, `npm audit`, Trivy
Container image scanning	Vulnerable OS packages and libraries in images	Trivy, Grype, ECR scanning

Scan	What it finds	Tools
IaC scanning	Insecure Terraform/K8s config (open SGs, public buckets)	Checkov, tfsec/Trivy config, kube-linter
DAST	Vulnerabilities in the running app	OWASP ZAP
SBOM	List of everything inside an image/app	Syft, Trivy

14.4 Secure CI/CD

- Protected branches: PR review + required status checks.
- Minimal pipeline permissions; OIDC instead of long-lived cloud keys.
- Pin actions/plugins/images to versions; keep Jenkins and plugins updated.
- Separate deploy credentials per environment; approvals for production.
- Fail the build on critical vulnerabilities and on leaked secrets.
- Sign images (cosign) and only deploy trusted images.

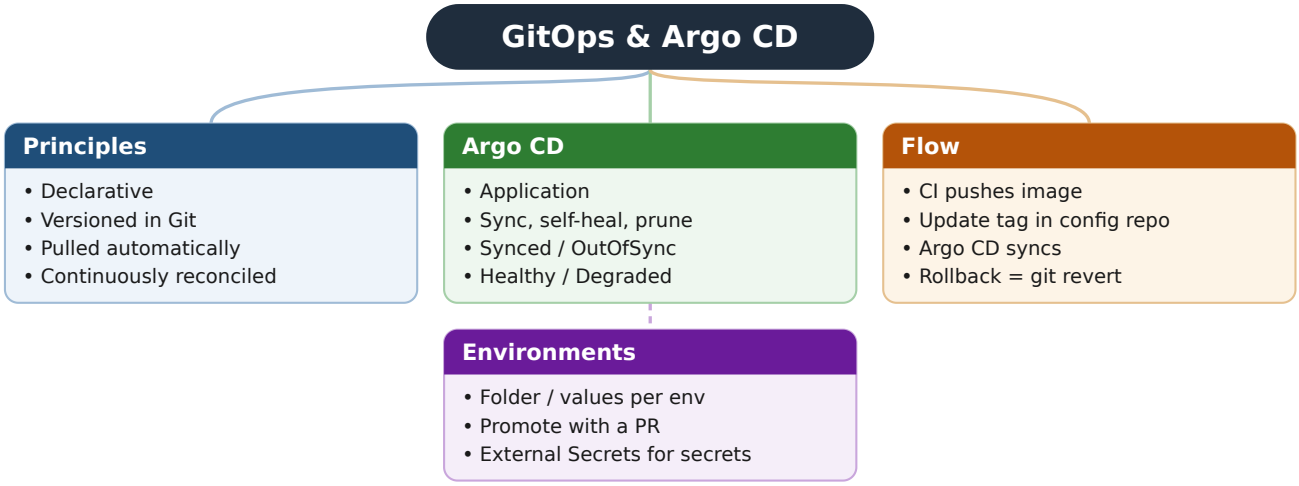
14.5 Security checklist by layer

Layer	Must do
Accounts / IAM	MFA, no root use, roles over keys, least privilege, CloudTrail on
Network	Private subnets for apps/DBs, tight SGs, no 0.0.0.0/0 on SSH/DB ports, TLS everywhere
Servers	Patched, SSH keys only, no root login, firewall, minimal packages
Containers	Non-root, minimal pinned images, scanned, no secrets inside
Kubernetes	RBAC, NetworkPolicies, Pod Security, secrets encryption, limits
Data	Encryption at rest (KMS) and in transit (TLS), backups tested
Code / pipeline	Scans, reviews, signed artifacts, secrets in a manager

REMEMBER

- Never in Git: **passwords, keys, tokens, .env files, tfstate**.
- Security is every layer: IAM → network → server → container → Kubernetes → data.

15. GitOps & Argo CD



15.1 GitOps principles

1. **Declarative:** the whole desired state is described in files (YAML/Helm/Kustomize).
2. **Versioned in Git:** Git is the single source of truth, with history and review.
3. **Pulled automatically:** an agent in the cluster pulls changes from Git (no `kubectl apply` from laptops or CI).
4. **Continuously reconciled:** the agent fixes any drift so the cluster always matches Git.

```
Developer → PR to app repo → CI builds & pushes image:1.5.0
          → CI (or a bot) updates image tag in the config repo (PR)
          → Argo CD sees the change in Git → syncs the cluster
Rollback  = git revert in the config repo
```

15.2 Argo CD

Term	Meaning
Application	Links a Git path (repo, branch, folder/chart) to a cluster + namespace
Sync	Apply Git's state to the cluster
Synced / OutOfSync	Cluster matches Git / differs
Healthy / Degraded	App's resources are working / failing
Auto-sync, self-heal, prune	Sync automatically, undo manual changes, delete removed resources
App of Apps / ApplicationSet	Manage many Applications from one place / generate them per environment or cluster

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: web-dev
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/<user>/platform-config
    targetRevision: main
    path: apps/web/overlays/dev          # or a Helm chart path with values
  destination:
    server: https://kubernetes.default.svc
    namespace: dev
  syncPolicy:
    automated: { prune: true, selfHeal: true }
    syncOptions: [CreateNamespace=true]
```

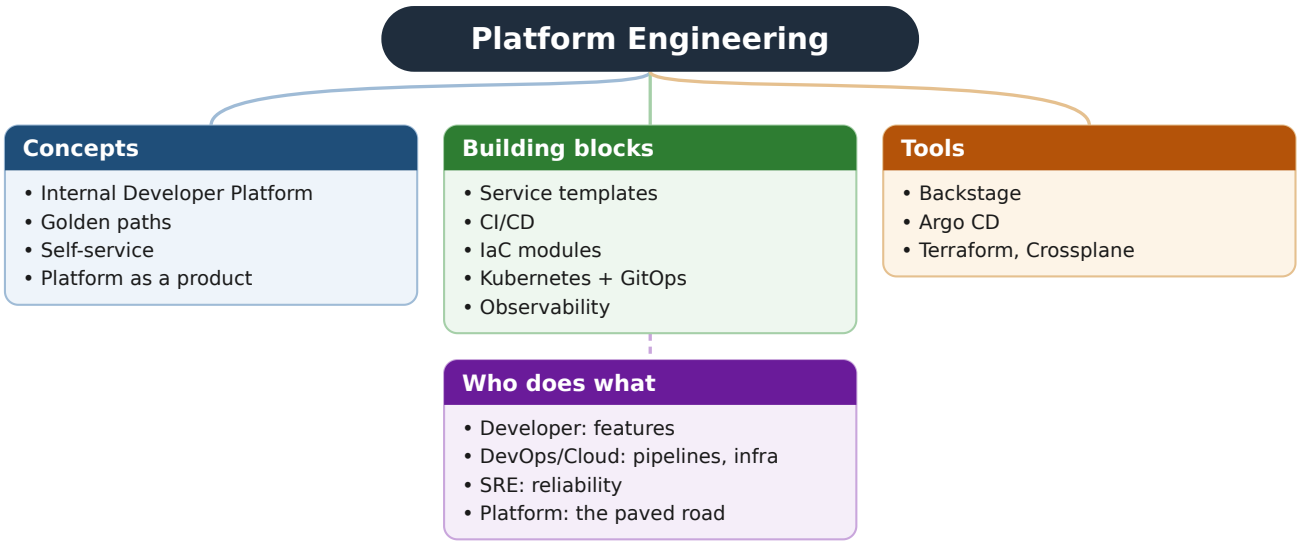
Command	What it does
`kubectl create ns argocd` + install manifest / Helm chart	Install Argo CD
`kubectl port-forward svc/argocd-server -n argocd 8080:443`	Open the UI
`argocd login localhost:8080`	CLI login
`argocd app list`, `argocd app sync web-dev`, `argocd app history web-dev`	Manage apps

Environments: one folder (or values file) per environment; promote from dev → prod with a pull request. **Secrets in GitOps:** never plain Secrets in Git. Use External Secrets Operator (reads from AWS Secrets Manager) or Sealed Secrets.

REMEMBER

- **Git is the source of truth.** Nobody runs `kubectl apply` on production by hand.

16. Platform Engineering



16.1 Concepts

Term	Meaning
Platform engineering	Building internal tools and paved roads so developers can ship safely by themselves
Internal Developer Platform (IDP)	The self-service layer: templates, pipelines, environments, infrastructure, observability — all wired together
Developer experience (DevEx)	How easy and fast it is for developers to do their work
Self-service	Developers create what they need (new service, database, environment) without opening tickets
Golden path	The recommended, pre-built, supported way to do a common task (e.g. "create a new web service")
Platform as a product	The platform team treats developers as customers: docs, feedback, roadmap
Platform API	The interface developers use: a portal, a CLI, Git templates, or Kubernetes custom resources
Backstage	Open-source developer portal: software catalog, templates (scaffolder), docs (TechDocs)
Crossplane	Lets you create cloud infrastructure through Kubernetes-style APIs

16.2 DevOps vs SRE vs Platform Engineering

Role	Main focus
Developer	Builds application features
DevOps engineer	Automation, CI/CD, infrastructure, culture of shared ownership between dev and ops
Cloud engineer	Designs and runs cloud infrastructure (networks, IAM, compute, cost)

Role	Main focus
SRE	Reliability: SLOs, monitoring, incident response, reducing toil with engineering
Platform engineer	Builds the internal platform and golden paths that the other roles use

```
Developer ──uses──► Internal Developer Platform (portal + templates + pipelines)
                          │ built and run by Platform team
                          ├── CI/CD (GitHub Actions)        ← DevOps practices
                          ├── IaC modules (Terraform)       ← Cloud engineering
                          ├── Kubernetes + Argo CD (GitOps)
                          └── Monitoring, SLOs, on-call      ← SRE
```

16.3 What a golden path template contains

- Repo skeleton (app code, Dockerfile, tests, README).
- CI pipeline (build, test, scan, push).
- Helm chart / Kubernetes manifests with sensible defaults (probes, limits, non-root).
- Argo CD Application per environment.
- Monitoring dashboards and alerts.
- Security defaults (secrets via External Secrets, least-privilege IAM role).

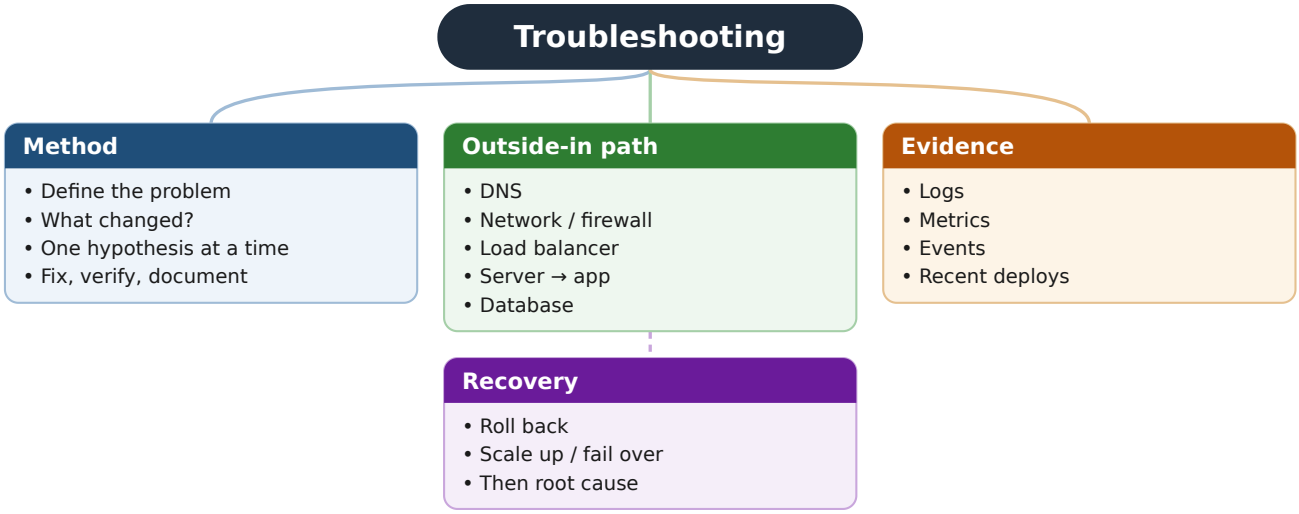
16.4 Final project reference architecture

```
GitHub (app repo) ──► GitHub Actions: test → scan → build → push ──► Amazon ECR
                                          │ updates image tag
GitHub (config repo: Helm values) ◄──────┘
        │ watched by
Argo CD (in EKS) ──► deploys Helm chart ──► App pods (EKS, private subnets)
                                                  │
AWS (built with Terraform): VPC, subnets, NAT, EKS, ECR, IAM roles, ALB
Observability: Prometheus + Grafana (+ CloudWatch) — alerts on SLOs
```

REMEMBER

- A good platform makes **the right way the easy way**.

17. Troubleshooting Playbooks



Universal method: 1) Define the problem exactly. 2) What changed? 3) Check from the outside in (DNS → network → load balancer → server → app → database). 4) Read logs and metrics. 5) Form one hypothesis, test it. 6) Fix, verify, document.

Scenario	Investigate in this order
Website is down	DNS resolves? (`dig`) → port open? (`curl -v`, `nc -zv`) → load balancer target health → server up? → service running? (`systemctl status`) → app logs → recent deploy? → database reachable?
DNS not resolving	`dig name` vs `dig @8.8.8.8 name` → record exists/correct in hosted zone? → TTL/caching → `/etc/resolv.conf` → typo / wrong domain
Server can't reach the internet	IP + gateway (`ip a`, `ip route`) → `ping 8.8.8.8` (network) vs `ping google.com` (DNS) → AWS: route to IGW/NAT, NAT in public subnet, SG/NACL outbound
EC2 unreachable	Instance state + status checks → public IP/EIP → SG inbound 22/80 from your IP → NACL → route table IGW → key/username → SSM Session Manager as a fallback
High CPU	`top` / `htop` → which process? → app logs → traffic spike or loop/bug? → scale out or fix → set alerts
Disk is full	`df -h` (and `df -i`) → `du -sh` to find big directories → logs, journal, Docker images, temp files → clean safely → add logrotate/alerts/bigger disk
App can't connect to database	Correct host/port/credentials (from secrets)? → `nc -zv db 5432` → SG allows app SG → DB running/available → max connections → TLS settings
Docker container keeps restarting	`docker ps -a` exit code → `docker logs` → `docker inspect` (OOMKilled) → env/config/volumes → run interactively to test
Pod CrashLoopBackOff	`kubectl describe pod` (Events, Last State) → `kubectl logs --previous` → missing ConfigMap/Secret/env → wrong command → probe killing it → memory limit
Deployment failed / not rolling out	`kubectl rollout status` → new pods' status (ImagePull? Crash? Pending?) → `kubectl rollout undo` if needed

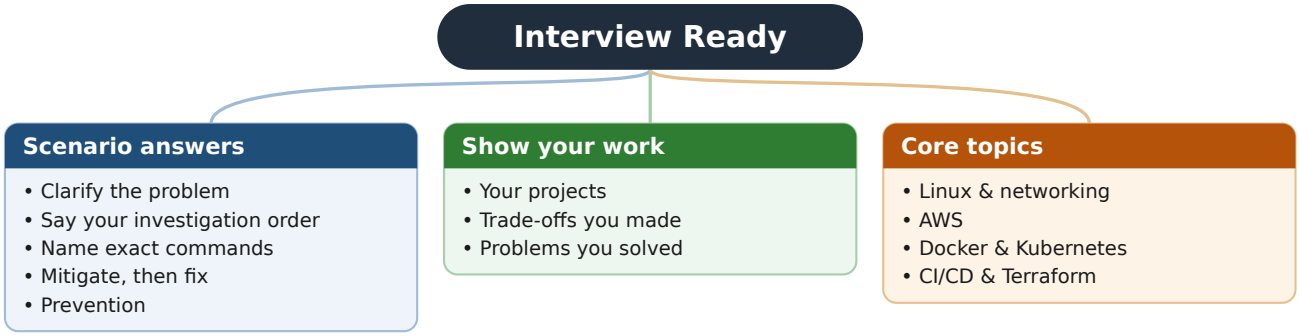
Scenario	Investigate in this order
Pipeline failed	Failing stage → first error in logs → code vs environment vs credentials vs network → reproduce locally → fix and re-run
Terraform apply failed	Read the exact error → credentials/region (`aws sts get-caller-identity`) → state lock → resource exists → quotas → dependency order → re-plan
Slow application	Latency dashboard → which endpoint → traces (which step) → DB slow queries → CPU/memory saturation → external dependency

Golden rules: change one thing at a time; keep notes; restore service first (rollback), then find the root cause; write a short blameless post-incident review.

REMEMBER

- The first question is always **"What changed?"**
- Change one thing at a time and write down what you did.

18. Interview Questions (Quick Answers)



18.1 Linux & networking

Question	Short answer
How do you find what is using port 8080?	`ss -tulpn | grep 8080` or `lsof -i :8080`
Disk is full — what do you do?	`df -h` → `du` to find big directories → clean logs/caches/images → check deleted-but-open files (`lsof +L1`) → add monitoring
Difference between a process and a service?	A process is a running program; a service is a process managed by systemd to run in the background
What does `chmod 755` mean?	Owner rwx, group r-x, others r-x
What is load average?	Average number of processes running or waiting for CPU over 1/5/15 minutes; compare with the CPU core count
What happens when you type a URL?	DNS → TCP handshake → TLS handshake → HTTP request → response → render
TCP vs UDP?	TCP reliable and ordered with a handshake; UDP fast with no guarantees
Private vs public subnet?	Public has a route to an Internet Gateway; private does not

18.2 Git & CI/CD

Question	Short answer
Merge vs rebase?	Merge keeps history with a merge commit; rebase rewrites commits into a straight line
How do you undo a pushed commit?	`git revert <hash>` (safe for shared branches)
CI vs CD?	CI builds and tests every change; CD automatically delivers/deploys passing changes
How do you store secrets in pipelines?	In the CI secret store or a secrets manager; prefer OIDC short-lived credentials over stored keys
Blue/green vs canary?	Blue/green switches all traffic at once between two environments; canary shifts traffic gradually

Question	Short answer
A pipeline failed — what do you do?	Find the failing stage, read the first error, reproduce locally, fix the cause, re-run

18.3 AWS

Question	Short answer
Security Group vs NACL?	SG: stateful, instance level, allow only. NACL: stateless, subnet level, allow + deny
IAM user vs role?	User has long-term credentials; role is assumed for temporary credentials
How does a private instance reach the internet?	Route `0.0.0.0/0` to a NAT Gateway in a public subnet
Multi-AZ vs read replica?	Multi-AZ = high availability (standby); read replica = scaling reads
How do you make a web app highly available?	ALB across 2+ AZs, Auto Scaling Group, Multi-AZ database, health checks
How do you control AWS costs?	Budgets/alerts, tags, right-sizing, Savings Plans/Spot, delete unused resources
EC2 is unreachable — what do you check?	State and status checks, public IP, SG, NACL, route table, key/user

18.4 Docker & Kubernetes

Question	Short answer
Image vs container?	Image = template; container = running instance of it
CMD vs ENTRYPOINT?	ENTRYPOINT = fixed command; CMD = default arguments, easy to override
How do you reduce image size?	Slim/distroless base, multi-stage builds, `.dockerignore`, fewer layers
Pod vs Deployment?	Pod = running containers; Deployment manages replicas, rolling updates and rollbacks
Liveness vs readiness probe?	Liveness restarts unhealthy containers; readiness controls whether traffic is sent
Pod in CrashLoopBackOff — steps?	`describe` (events, last state) → `logs --previous` → check config/secrets/command/limits
How do Services find pods?	By label selectors; check with `kubectl get endpoints`
ConfigMap vs Secret?	ConfigMap for normal config; Secret for sensitive data (needs encryption + RBAC)
Requests vs limits?	Requests = reserved for scheduling; limits = maximum allowed

18.5 Terraform, monitoring, GitOps, platform

Question	Short answer
What is Terraform state and why remote?	Record of managed resources; remote (S3 + locking) so teams share it safely
`plan` vs `apply`?	Plan previews changes; apply makes them
What is drift?	Real infrastructure differs from code (manual changes); detect with `plan`
Metrics vs logs vs traces?	Numbers over time / event records / request path across services
SLI vs SLO vs SLA?	What you measure / target / contractual promise
What is GitOps?	Git is the source of truth; an in-cluster agent (Argo CD) pulls and reconciles
What is a golden path?	The supported, pre-built way for developers to do a common task
DevOps vs Platform Engineering?	DevOps is the culture and practices; platform engineering builds self-service tools that make them easy

18.6 How to answer scenario questions

1. Clarify: what exactly is failing, for whom, since when, what changed?
2. Say your investigation order out loud (outside in: DNS → network → load balancer → server → app → database).
3. Name the exact commands or consoles you would use.
4. Mitigate first (rollback/scale), then find the root cause.
5. Finish with prevention: monitoring, alerts, automation, documentation.

REMEMBER

- Behavioral questions: **STAR** = Situation, Task, Action, Result.
- "I don't know, but here is how I would find out" is a good answer.

Revision History

Version	Date	What changed
1.0	17 Oct 2026	First complete edition: 17 sections from computer basics to platform engineering
1.1	17 Oct 2026	Gap review: binary and subnetting method, deployment strategies, SRE and incident management, Kubernetes scheduling/NetworkPolicy/Kustomize, AWS multi-account and security services, extra Git/Bash/Python/Docker/Terraform/Ansible features, new section 18 Interview Q&A
1.2	17 Oct 2026	Visual aids added: big-picture page, a mind map at the start of every section, a REMEMBER box at the end of every section. No content changed
1.3	17 Oct 2026	New syntax guides: 5.7 Bash syntax, 6.7 Python basics, 9.8 Terraform language (HCL), 10.1 YAML and 10.2 Ansible playbook anatomy. All code examples tested (Bash run, Python run, `terraform validate`, `ansible-playbook --syntax-check`). Existing Terraform examples re-spaced with `terraform fmt` (formatting only)
1.4	17 Oct 2026	Split into 18 separate topic PDFs (notes/topics/) for printing one topic at a time. Content unchanged